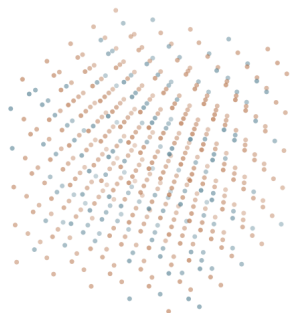


# AtomForge

## Detailed user and reference manual

Building structures, exploring atomic geometry, and analysing electronic fields

Cu-Ni substitutional alloy  
500 atoms



Cu nanosphere  
336 atoms



Cu Sigma-5 boundary  
60 atoms



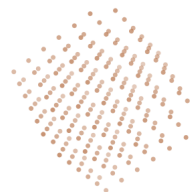
Cu Voronoi polycrystal  
1,329 atoms



Si-O random packing  
120 atoms



Cu mesh fill  
224 atoms



Documentation edition: 9 September 2026

Application source baseline: 0df0af2.

Includes reproducible examples, a CHGCAR walkthrough, desktop controls, command-line recipes, and Python integration.

# How to use this manual

Start with Chapter 1 to open a file, construct a crystal, and save a result. Chapters 2–4 explain the structural workflows. Chapters 5–7 cover electronic loading, visualisation, charge redistribution, and the complete electronic operation catalogue. Chapter 8 provides scripting recipes; the appendices cover installation, reproducibility, and troubleshooting.

Menu names and controls describe the source baseline above and the Windows executable used to prepare the figures. Features controlled by build options are marked explicitly. A source file does not by itself mean a feature is accessible in the desktop menu. Common-neighbour analysis, angular-distribution analysis, and the interface builder are not exposed in the baseline menus; this manual does not present them as usable menu commands.

GUI screenshots are captured from AtomForge. The builder gallery and CHGCAR plots are separately rendered scientific figures calculated from real AtomForge outputs; they are not interface mockups. Example structures are starting configurations, not validated equilibrium structures. The supplied CHGCAR is used to demonstrate the software; its physical system and calculation settings were not independently established.

All paths beginning with `examples/` are relative to this manual's folder. Commands use `AtomForge` for readability; on Windows substitute the path to `AtomForge.exe`. Long command listings may wrap visually; each displayed command is entered as one shell command. Python examples assume the package and native electronic library are available as described in Chapter 8.

# Contents

|                                                                      |           |
|----------------------------------------------------------------------|-----------|
| <b>How to use this manual</b>                                        | <b>i</b>  |
| <b>1 Getting started</b>                                             | <b>1</b>  |
| 1.1 What AtomForge does . . . . .                                    | 1         |
| 1.2 First structure: a copper crystal . . . . .                      | 1         |
| 1.3 Opening, saving, and browsing . . . . .                          | 2         |
| 1.4 File types and what survives export . . . . .                    | 2         |
| 1.5 Drag and drop . . . . .                                          | 2         |
| 1.6 Orienting and inspecting the main view . . . . .                 | 3         |
| 1.7 A productive project layout . . . . .                            | 3         |
| <b>2 Building atomic structures</b>                                  | <b>4</b>  |
| 2.1 Common builder workflow . . . . .                                | 4         |
| 2.2 Bulk Crystal . . . . .                                           | 4         |
| 2.3 Substitutional Solid Solution . . . . .                          | 5         |
| 2.4 CSL Grain Boundary . . . . .                                     | 6         |
| 2.5 Nanocrystal: shape cuts . . . . .                                | 6         |
| 2.6 Nanocrystal: Wulff construction . . . . .                        | 7         |
| 2.7 Custom Structure from OBJ/STL . . . . .                          | 7         |
| 2.8 Polycrystal . . . . .                                            | 8         |
| 2.9 Amorphous Structure . . . . .                                    | 8         |
| 2.10 Optional and unavailable builder menus . . . . .                | 8         |
| 2.11 Worked study: compare bulk and finite Cu environments . . . . . | 9         |
| <b>3 Editing, assembly, and defects</b>                              | <b>10</b> |
| 3.1 Selection and edit history . . . . .                             | 10        |
| 3.2 Edit Structure: atoms and cell . . . . .                         | 10        |
| 3.3 Transform Structure . . . . .                                    | 11        |
| 3.4 Merge Structures . . . . .                                       | 11        |
| 3.5 Cell Sculptor: supercells and cutting slabs . . . . .            | 11        |
| 3.6 Insert Dislocation . . . . .                                     | 12        |
| 3.7 Add Interstitial Atoms and detect voids . . . . .                | 12        |

|          |                                                                |           |
|----------|----------------------------------------------------------------|-----------|
| 3.8      | Checking an edited model . . . . .                             | 13        |
| <b>4</b> | <b>Structural analysis and presentation</b>                    | <b>14</b> |
| 4.1      | Structure Info and Atom Info . . . . .                         | 14        |
| 4.2      | Distance and angle measurements . . . . .                      | 14        |
| 4.3      | Radial Distribution Function . . . . .                         | 14        |
| 4.4      | Short-Range Order (SRO) . . . . .                              | 15        |
| 4.5      | Lattice planes and Miller directions . . . . .                 | 15        |
| 4.6      | Voronoi and coordination polyhedra . . . . .                   | 16        |
| 4.7      | Atomic sizes, colours, and lighting . . . . .                  | 16        |
| 4.8      | Exporting images . . . . .                                     | 16        |
| 4.9      | Features that are not current analysis-menu commands . . . . . | 16        |
| <b>5</b> | <b>Electronic files and interactive views</b>                  | <b>17</b> |
| 5.1      | Open a scalar field . . . . .                                  | 17        |
| 5.2      | VASP, Cube, and XSF conventions . . . . .                      | 17        |
| 5.3      | Units, sampling, and periodic endpoints . . . . .              | 18        |
| 5.4      | What does Reference mean? . . . . .                            | 18        |
| 5.5      | 3D volume and isosurface views . . . . .                       | 19        |
| 5.6      | Choosing an initial level and understanding warnings . . . . . | 20        |
| 5.7      | 2D lattice-plane sections . . . . .                            | 20        |
| 5.8      | Arbitrary planes and the 3D guide . . . . .                    | 20        |
| 5.9      | Worked CHGCAR walkthrough . . . . .                            | 21        |
| 5.10     | Exporting the right result . . . . .                           | 22        |
| <b>6</b> | <b>Charge redistribution and Boolean masks</b>                 | <b>23</b> |
| 6.1      | Define the study before subtracting . . . . .                  | 23        |
| 6.2      | Grid compatibility . . . . .                                   | 23        |
| 6.3      | Weighted density difference: complete GUI recipe . . . . .     | 23        |
| 6.4      | Accumulation and depletion fields . . . . .                    | 24        |
| 6.5      | Charge summary . . . . .                                       | 24        |
| 6.6      | Threshold mask . . . . .                                       | 24        |
| 6.7      | Boolean masks and inversion . . . . .                          | 25        |
| 6.8      | Apply mask and quantify a region . . . . .                     | 25        |
| 6.9      | Cumulative charge profile . . . . .                            | 25        |
| 6.10     | Recommended record for a charge study . . . . .                | 26        |
| <b>7</b> | <b>Electronic operation reference</b>                          | <b>27</b> |
| 7.1      | How to run an operation . . . . .                              | 27        |
| 7.2      | Total integral . . . . .                                       | 27        |
| 7.3      | Add and subtract reference . . . . .                           | 27        |

|          |                                                            |           |
|----------|------------------------------------------------------------|-----------|
| 7.4      | Multiply and divide reference . . . . .                    | 27        |
| 7.5      | Scale . . . . .                                            | 28        |
| 7.6      | Gaussian smoothing . . . . .                               | 28        |
| 7.7      | Cartesian gradient . . . . .                               | 28        |
| 7.8      | Laplacian . . . . .                                        | 28        |
| 7.9      | Energy-density conversion . . . . .                        | 28        |
| 7.10     | Line profile . . . . .                                     | 29        |
| 7.11     | Planar average . . . . .                                   | 29        |
| 7.12     | Macroscopic average . . . . .                              | 29        |
| 7.13     | Numerical plane section . . . . .                          | 29        |
| 7.14     | Contour segments . . . . .                                 | 30        |
| 7.15     | Peak search . . . . .                                      | 30        |
| 7.16     | Voronoi integration . . . . .                              | 30        |
| 7.17     | Sphere integration . . . . .                               | 30        |
| 7.18     | Structure factors from a grid . . . . .                    | 31        |
| 7.19     | Fourier synthesis . . . . .                                | 31        |
| 7.20     | Patterson map . . . . .                                    | 31        |
| 7.21     | Ewald point-charge electrostatics . . . . .                | 31        |
| 7.22     | Isosurface extraction and reference colouring . . . . .    | 32        |
| 7.23     | Resample to reference grid . . . . .                       | 32        |
| 7.24     | Verify periodic endpoint planes . . . . .                  | 32        |
| <b>8</b> | <b>Python and command-line integration</b>                 | <b>33</b> |
| 8.1      | Install the Python package . . . . .                       | 33        |
| 8.2      | Structure objects and file I/O . . . . .                   | 33        |
| 8.3      | Desktop and notebook viewing . . . . .                     | 34        |
| 8.4      | Loading electronic volumes . . . . .                       | 34        |
| 8.5      | A complete scripted charge workflow . . . . .              | 35        |
| 8.6      | Profiles, sections, derivatives, and integration . . . . . | 35        |
| 8.7      | Fourier and scattering models . . . . .                    | 36        |
| 8.8      | Point charges, surfaces, and explicit geometry . . . . .   | 36        |
| 8.9      | Electronic CLI . . . . .                                   | 37        |
| 8.10     | Native builder CLI . . . . .                               | 37        |
| <b>A</b> | <b>Installation, builds, and troubleshooting</b>           | <b>38</b> |
| A.1      | Running a packaged desktop build . . . . .                 | 38        |
| A.2      | Build requirements . . . . .                               | 38        |
| A.3      | Linux build . . . . .                                      | 39        |
| A.4      | Windows build with MSYS2 UCRT64 . . . . .                  | 39        |
| A.5      | Portable packaging and CI . . . . .                        | 39        |

|          |                                                          |           |
|----------|----------------------------------------------------------|-----------|
| A.6      | Reporting a reproducible problem . . . . .               | 39        |
| A.7      | Troubleshooting by symptom . . . . .                     | 40        |
| <b>B</b> | <b>Examples, validation, and maintaining this manual</b> | <b>41</b> |
| B.1      | Included example inventory . . . . .                     | 41        |
| B.2      | CHGCAR provenance . . . . .                              | 41        |
| B.3      | Regenerate examples and figures . . . . .                | 42        |
| B.4      | Compile the LaTeX manual . . . . .                       | 42        |
| B.5      | Documentation coverage and limitations . . . . .         | 42        |
| B.6      | Source map for maintainers . . . . .                     | 43        |
| B.7      | Glossary . . . . .                                       | 43        |

# Chapter 1

## Getting started

### 1.1 What AtomForge does

AtomForge is a desktop structure builder and viewer with a native electronic post-processing engine. It constructs atomistic starting configurations, edits positions and cells, renders structures, measures geometric quantities, and processes scalar fields from electronic-structure calculations. It does not run a self-consistent electronic calculation, relax a structure, or predict a formation energy merely by building a model.

The main structure workspace and the electronic dialog maintain different data. Opening a CIF in the main workspace supplies atom positions and a cell. Opening CHGCAR in **Analysis > Electronic Post-processing** supplies a sampled scalar field and imported sites. A structure alone does not contain a charge density; a rendered mesh alone does not contain an electronic wavefunction.

### 1.2 First structure: a copper crystal

1. Start the executable. Choose **Build > Bulk Crystal**. Select the cubic crystal system and space group 225.
2. Enter  $a = 3.61 \text{ \AA}$ . Use a Cu site at fractional coordinates (0,0,0) in the asymmetric-unit list. Remove unrelated default sites if present; do not add the three symmetry-equivalent FCC positions yourself.
3. Build the crystal. Inspect the generated structure and its cell. The supplied example `examples/cu_fcc.cif` has four Cu atoms in the conventional cubic cell.
4. Close the builder when finished. Use **View > Structure Info** to inspect counts and lattice information.
5. Choose **File > Save As**, browse to your work folder, enter a new filename such as `cu_first.cif`, select CIF, and save. Reopen the saved file to check that the cell and species are retained.

If symmetry support is unavailable in a custom build, install/build with spglib or open the supplied expanded CIF instead. The lattice parameter is an example input; select a parameter appropriate to your intended temperature, composition, and calculation method.

## 1.3 Opening, saving, and browsing

**File > Open** reads a structure into the workspace; **Open Recent** returns to previously used paths. The browser supports directory navigation and file selection. Double-click a directory to enter it, select a file, then use the dialog's opening action. The location field is useful for jumping to a directory; it is not necessary to type the entire file path.

**Save** writes the active structure to its current destination. Use **Save As** to preserve the original and choose a new filename/format. A structure save records supported structural data, not every display setting or electronic intermediate. Save derived electronic grids separately from the electronic dialog. **Close** closes the current structure; **Quit** exits the program. Save valuable edits before closing.

| Task                   | Control or shortcut                                                                                |
|------------------------|----------------------------------------------------------------------------------------------------|
| Open a structure       | <b>File &gt; Open</b> , Ctrl+O.                                                                    |
| Save active structure  | <b>File &gt; Save</b> , Ctrl+S.                                                                    |
| Save a new copy        | <b>File &gt; Save As</b> , Ctrl+Shift+S.                                                           |
| Export rendered image  | <b>File &gt; Export Image</b> , Ctrl+Alt+S.                                                        |
| Close active structure | <b>File &gt; Close</b> , Ctrl+W.                                                                   |
| Undo / redo            | <b>Edit &gt; Undo</b> and <b>Edit &gt; Redo</b> ; availability follows the edit history.           |
| Electronic analysis    | <b>Analysis &gt; Electronic Post-processing</b> ; available without a structure in the main scene. |
| Appearance controls    | <b>Settings &gt; Atomic Sizes, Display Settings, Polyhedral Settings</b> .                         |

## 1.4 File types and what survives export

The desktop uses Open Babel for structure I/O. Its advertised structure extensions include XYZ, CIF, PDB, SDF, MOL, VASP, MOL2, PWI, and GJF. Actual conversion depends on the installed Open Babel plugins. Select a format for the information you need: CIF or VASP for periodic cells, XYZ for simple coordinate interchange, and a molecular format for molecular connectivity where supported.

Plain XYZ does not reliably communicate a periodic lattice to every downstream reader. PDB has format-specific field limits. Format conversions may not preserve custom atom colours, grain identifiers, overlays, or all calculation metadata. Check the exported file in the program that will consume it, especially after transforming a non-orthogonal cell.

Electronic inputs are a separate family: VASP charge/potential/ELF grids, Gaussian Cube, and XSF datagrids. Electronic outputs include XSF, Cube, and VASP grids; CSV tables; and OBJ/PLY isosurface meshes. A CSV profile is a numerical table, while an OBJ surface is geometry. Neither is a replacement for saving the full scalar grid.

## 1.5 Drag and drop

With the electronic dialog open, drop a supported volumetric file onto the application to load it there. Enable **Drop files as reference** before dropping an operand density. Files arriving during a calculation are queued; wait for the active calculation to finish before interpreting the new field selection. A reference-file drop supplies the second input; it does not subtract anything until you choose and calculate an operation.



Several structure builders accept dropped structures or meshes in their own preview/source regions. Use the visible drop guidance in the active dialog. For an unambiguous workflow, open only the builder you intend to supply and verify the displayed source filename and atom count immediately afterwards.

## 1.6 Orienting and inspecting the main view

Use the top toolbar's X, Y, Z buttons for Cartesian-axis views and a, b, c for lattice-direction views when a cell is available. The rotation step field and adjacent axis buttons apply a repeatable view rotation. **View > Reset Default View** restores framing; it does not undo atom edits. **Isometric View** and **Orthographic View** change the viewing mode. Use orthographic views when comparing projected distances or alignments.

**Show Atoms** and **Show Bounding Box** control geometry visibility. **Show Element** toggles element labels. **View Structure By** switches available colouring between elements, crystal orientation, and grain-boundary information. Orientation/boundary colouring requires corresponding structure metadata; a plain XYZ file may not contain it.

**View Atoms By** offers balls, ball-and-stick, space-filling, and polyhedral representations when supported by the current state. Bond-like lines and coordination polyhedra are visual/geometric constructs based on neighbourhood criteria; do not infer a quantum-mechanical bond order from their appearance.

## 1.7 A productive project layout

Create separate folders for input calculations, generated structures, derived fields, tables, and figures. Keep original electronic outputs unchanged. Name derived fields by operation, for example **AB\_minus\_A\_minus\_B.xsf**, and record the reference weights, grid geometry, selected channel, and units beside numerical results. For publication, retain the actual density difference and quantitative tables as well as the screenshot.

The folder containing this manual follows that pattern on a smaller scale: **chapters/** holds editable LaTeX, **figures/** holds published illustrations, **examples/** holds reusable structures and tables, **scripts/** holds regeneration tools, and **build/** holds disposable LaTeX/render intermediates.

## Chapter 2

# Building atomic structures

### 2.1 Common builder workflow

Choose the appropriate command under **Build**, supply a periodic reference where required, set parameters, inspect the preview, and run the build action. Read status messages before saving the result. Source previews and generated previews serve different purposes: changing a parameter may update an illustrative preview while the atomic result still needs rebuilding.

Use modest cells first. Replicating a structure multiplies its atom count; a  $10 \times 10 \times 10$  replication of a four-atom cell contains 4,000 atoms before cuts. Increasing all three dimensions by two multiplies the workload by eight. Once the geometry is correct, increase size and inspect boundary contacts. A builder does not automatically relax strained bonds or remove every physically undesirable local environment.

### 2.2 Bulk Crystal

**Purpose.** Generate a symmetry-expanded periodic crystal from lattice parameters and asymmetric-unit sites. Inputs are the crystal system, space group, cell dimensions/angles, and element/fractional-coordinate rows.

**Procedure.** Select one of the seven crystal systems and a compatible space group (1–230). Set independent lattice parameters; check constraints implied by the selected system. Add each crystallographically independent site using an element and direct coordinates. Use the row deletion or clear controls to remove unwanted defaults. Build, inspect the expanded sites, and save a periodic format.

Fractional coordinates are coefficients in  $\mathbf{r} = f_a\mathbf{a} + f_b\mathbf{b} + f_c\mathbf{c}$ . They are not distances in Å. Equivalent positions are generated by symmetry; entering a full conventional basis as though every site were independent can duplicate sites. Inspect the final count and short contacts.

**Worked examples.** FCC Cu with group 225,  $a = 3.61$  Å, and Cu at (0,0,0) gives four atoms. BCC Fe with group 229,  $a = 2.87$  Å, and Fe at (0,0,0) gives two. These examples exercise symmetry expansion; they do not determine a thermodynamically optimal lattice parameter.

```
AtomForge --build bulk --system cubic --spacegroup 225 --a 3.61 --atom "Cu 0 0 0" --  
output cu_fcc.cif  
AtomForge --build bulk --system cubic --spacegroup 229 --a 2.87 --atom "Fe 0 0 0" --  
output fe_bcc.cif
```

For lower-symmetry cells, set `--b`, `--c`, and angles `--alpha`, `--beta`, `--gamma`. Repeat `--atom` for independent sites. See `--help` for accepted system names.

Figure 2.1: The Bulk Crystal dialog at its initial settings. Change the default H site to Cu and the default lattice parameter to 3.61 Angstrom for the FCC Cu walkthrough. The element button opens a periodic-table picker.

## 2.3 Substitutional Solid Solution

**Purpose.** Assign species to existing lattice sites with requested fractions. This changes chemical labels on a host structure; it does not search for a special quasirandom structure or optimise chemical short-range order.

**Procedure.** Open a host with enough sites to represent the composition, then choose **Build > Substitutional Solid Solution**. Select the source and species fractions. Fractions must describe the intended whole assignment and sum to one. Choose a fixed seed for reproducibility. Generate the alloy and check the resulting species counts before export. For small hosts, integer atom counts limit the achievable composition.

The example host is a  $5 \times 5 \times 5$  repetition of conventional FCC Cu: 500 sites. The 70% Cu / 30% Ni assignment uses seed 42. Keep the same host atom ordering and seed when reproducing an assignment; changing the host can change which particular sites are substituted.

```
AtomForge --build sss --input cu_host.cif --frac "Cu=0.7,Ni=0.3" --seed 42 --output
cu_ni_alloy.cif
```

The menu is controlled by `ATOMFORGE_ENABLE_SSS_BUILDER`, enabled by default. A seed of zero in the CLI requests a time-dependent seed. After substitution, an external relaxation may be needed to account for atomic size mismatch.

## 2.4 CSL Grain Boundary

**Purpose.** Construct a coincidence-site-lattice bicrystal from a compatible cubic periodic reference. A coincidence index  $\Sigma$  describes lattice coincidence, not a calculated grain-boundary energy.

**Procedure.** Load the reference cell in the grain-boundary dialog. Enter a rotation axis  $[u\ v\ w]$  and select a supported  $\Sigma$  and candidate boundary plane. Set the thickness/repetition of each grain, any gap and vacuum, and an overlap-removal distance. Build and inspect both the interface and periodic cell boundaries. Save the bicrystal and relax it externally before comparing energies.

**Controls.** The axis is a crystallographic direction. The plane selector chooses from generated candidates. The CLI's `--plane` is a zero-based candidate index, not a Miller index. `--uca` and `--ucb` control stacking repetitions for the two grains. `--gap` separates the grains; `--vacuum` adds vacuum on each side. `--overlap` removes close contacts using a distance in Å; an excessive cutoff can delete desirable interfacial atoms.

```
AtomForge --build gb --input cu_fcc.cif --axis "0 0 1" --sigma 5 --plane 0 --uca 3 --
ucb 3 --overlap 1.5 --output cu_sigma5.cif
```

This manual's generated example contains 60 atoms. The small thickness makes it a geometry demonstration, not a converged boundary-energy supercell. If `--sigma` is omitted, candidate search uses `--sigmax`; specify the exact value when reproducibility matters. `--conventional` skips the primitive-cell conversion. Check both lattice compatibility and the selected candidate before interpreting a failed build as a file-format problem.

## 2.5 Nanocrystal: shape cuts

**Purpose.** Fill a finite shape with atoms from a periodic crystal. Load a periodic reference, select **Shape Cut**, choose a shape, set dimensions and centre, and use automatic replication unless you need explicit repetition counts. Build the nanocrystal and inspect the output bounding cell.

| Shape                | Main dimensions                                                              |
|----------------------|------------------------------------------------------------------------------|
| Sphere               | Radius.                                                                      |
| Ellipsoid            | Radii along the three local axes.                                            |
| Box                  | Half-lengths along the three axes; full side lengths are twice these values. |
| Cylinder             | Radius, full height, and cylinder axis.                                      |
| Octahedron           | Size parameter for the octahedral cut.                                       |
| Truncated octahedron | Size and truncation parameters.                                              |
| Cuboctahedron        | Size parameter for the cuboctahedral cut.                                    |

**Auto-center from atoms** derives a centre from the reference. Manual centring changes which lattice sites intersect the boundary and can change surface termination and atom count. **Auto-replicate** chooses enough source repetitions for the cut; manual repetitions that are too small truncate the intended object. **Rectangular cell** and vacuum settings provide an enclosing output cell; this does not make a nanoparticle a bulk material.

```
AtomForge --build nano --input cu_fcc.cif --shape sphere --radius 10 --vacuum 5 --
output cu_sphere.xyz
```

The example contains 336 atoms. Use `--rx/--ry/--rz` for ellipsoid radii, `--hx/--hy/--hz` for box half-lengths, and `--cylradius/--cylheight/--cylaxis` for the cylinder. Explicit source

replication uses `--repa`, `--repb`, and `--repc`; zero selects automatic sizing in the CLI.

## 2.6 Nanocrystal: Wulff construction

Select **Wulff Construction** in the nanocrystal dialog. Add facet families, each with nonzero ( $hkl$ ) indices and a positive relative surface energy. AtomForge expands families using the reference symmetry, previews the bounding planes, and cuts the periodic crystal by the resulting shape.

1. Load the periodic reference and check its orientation and symmetry.
2. Add, for example, (100) and (111) families with illustrative energies 1.0 and 0.9. These are demonstration ratios, not material data.
3. Set **Max radius (Å)**. The implementation scales plane distances by energy ratios, placing the highest-energy family at this maximum distance. This control is not necessarily the circumscribed vertex radius of the final polyhedron.
4. Inspect the **Wulff Plane View**; family colours help identify the contribution of each family. Remove redundant families or add missing ones as required.
5. Build the atomic nanocrystal. Inspect discrete surface termination separately from the continuous plane preview, then export.

The construction requires a bounded intersection of planes. A family may disappear from the final shape when other planes enclose it. Energy ratios must come from comparable surface calculations if a physical equilibrium-shape interpretation is intended. Atomic discretisation can break an ideal continuous shape's apparent symmetry in a very small particle. Wulff controls are documented here as a GUI workflow; the shape-cut CLI examples do not expose the same family editor.

## 2.7 Custom Structure from OBJ/STL

**Purpose.** Fill a closed mesh with lattice sites. The mesh is a spatial boundary, not a collection of atoms. Use a watertight OBJ or STL, a periodic atomic reference, a physical scale, and an orientation.

**Procedure.** Open **Build > Custom Structure**. Load/drop the reference and mesh into their corresponding inputs. Set the mesh scale in Å per model unit, then adjust rotation or the Miller-direction alignment. Select enough replication to cover the mesh, set vacuum, and build. Inspect thin features and the atom count; a feature narrower than the lattice spacing may contain no sites.

```
AtomForge --build custom --input cu_fcc.cif --mesh octahedron.obj --scale 12 --vacuum 5
--output cu_mesh.xyz
```

The included octahedron has six vertices at the positive and negative unit axes and eight triangular faces. At scale 12 it produces 224 Cu atoms in this example. No Blender installation is required. To use Blender, create a closed mesh, apply the intended object transforms before export, export OBJ/STL, and verify the scale in AtomForge. Inspect for holes, self-intersections, and reversed or inconsistent faces when the fill is missing or unexpected.

The CLI provides `--rotx`, `--roty`, and `--rotz` in degrees. `--miller

# "hkl"` aligns a crystallographic direction with  $+Z$  and overrides Euler orientation. A crystallographic direction  $[hkl]$  and a plane normal  $(hkl)$  differ in a general non-cubic cell; choose the alignment convention deliberately.

## 2.8 Polycrystal

**Purpose.** Populate a box with grains cut by a Voronoi partition. Inputs include a periodic reference, box dimensions, grain count, random seed, and optional orientations. The construction creates geometric grain boundaries; it does not optimise their energies.

**Procedure.** Load a reference, enter box lengths and grain count, set the seed, and choose orientations or random assignment. Generate, inspect the grain sizes and contacts, and save. Use orientation colouring when the output retains orientation metadata. Reopening a format that lacks grain metadata can preserve atoms while losing those colours.

```
AtomForge --build poly --input cu_fcc.cif --sizeX 25 --sizeY 25 --sizeZ 25 --grains 4
--seed 7 --output cu_polycrystal.cif
```

This example contains 1,329 atoms. The CLI accepts repeated `--euler"phi1Phi2"` triplets in Bunge convention, one per grain; unspecified orientations are random. Use the same seed, dimensions, reference, and ordering to reproduce a sample. Increase the box relative to grain count when grains become too small to have a meaningful crystalline interior.

## 2.9 Amorphous Structure

**Purpose.** Produce a random packing subject to distance constraints. Specify elements and counts, a mass density or explicit box, pair-distance constraints, and a seed. This is an initial configuration generator; random packing alone does not establish a realistic glass network.

**Procedure.** Add the required species/count rows. Set density in  $\text{g cm}^{-3}$  or explicit box dimensions. Check the minimum separation settings; overly strict cutoffs at a high density can make placement impossible. Set a fixed seed, generate, and inspect the reported inserted count and any placement failure. Relax or equilibrate the configuration with a suitable external method before structural or energetic claims.

```
AtomForge --build amorphous --element "Si 40" --element "O 80" --density 1.5 --seed 7
--output sio2_pack.xyz
```

The example has 120 atoms with a 1:2 Si:O ratio. The deliberately low packing density is useful for demonstrating construction and is not a claim about silica's equilibrium density. `--boxa/--boxb/--boxc` override density-derived dimensions. `--mindist"Z1Z2distance"` adds a pair constraint using atomic numbers; `--covtol` controls the covalent-radius fallback; `--attempts` bounds placement attempts. If placement fails, examine the geometry and constraints before merely raising the attempt limit.

## 2.10 Optional and unavailable builder menus

The stacking-fault builder is compiled into the desktop menu only with `ATOMFORGE_ENABLE_SFE_BUILDER=ON`; the default is OFF. Its purpose is to construct faulted configurations, not to calculate a stacking-fault energy. In an enabled build, open **Build > Stacking Faults**, supply a reference with **Use Current Scene** or the source workflow, and click **Detect Structure**. Select a supported Plane, choose **Smallest Unit Cell** or **Orthogonal Cell**, and set Layers (2–96). Click **Generate Sequence**, use the Structure slider to inspect its members, and choose **Use Selected Structure** to adopt one. **Export All Structures** writes the sequence to the specified Folder/Subfolder; **Use Source Folder** supplies a convenient base when available. Validate the stacking geometry and

calculate energies externally. This optional path was inspected in source but not enabled in the screenshot executable.

The interface-builder source exists but is not connected to the current main menu and dialog drawing sequence. Use the exposed merge/transform workflows for manual assembly rather than searching for an unavailable menu command.

## 2.11 Worked study: compare bulk and finite Cu environments

This exercise connects building, file inspection, and structural analysis. It uses only generated structures and does not require an electronic calculation.

1. Open `examples/cu_fcc.cif`. Inspect its four sites and cubic cell. The cell edge is 3.61 Å. Save a working copy before changing it.
2. Open the supplied `cu_host.cif`, or create it by repeating the conventional cell  $5 \times 5 \times 5$  through Python. It contains 500 atoms. Use this larger periodic host for bulk neighbour statistics.
3. Open `cu_sphere.xyz` separately. It contains 336 atoms from the same reference lattice, cut at radius 10 Å. Rotate it and inspect surface sites. The smaller count is caused by the finite shape, not by a different crystal symmetry.
4. Run RDF on the periodic host with Cu as both species, PBC enabled, and no smoothing. Check the first-neighbour peak near 2.553 Å. Begin with a cutoff that is comfortably smaller than the box dimensions.
5. Run RDF on the finite sphere with PBC disabled. Compare raw counts and cumulative coordination before comparing normalised curves: the particle has surface atoms with incomplete neighbour shells and its enclosing vacuum does not define a bulk atomic density.
6. Open `cu_ni_alloy.cif`. Verify 350 Cu and 150 Ni atoms, then inspect Cu–Cu and Cu–Ni partial distributions and the SRO table. The unrelaxed substitution changes labels while retaining ideal host positions; it cannot demonstrate size-mismatch relaxation.

Keep the source filenames, cutoffs, species choices, PBC setting, bin count, and smoothing with each plot. If the bulk nearest-neighbour peak moves unexpectedly, first check lattice scaling and units. If coordination falls at the surface of the sphere, inspect its geometric boundary before treating the change as disorder. If the alloy has nonzero measured SRO, compare seeds and finite-size effects before interpreting it as evidence of an energetic ordering preference.

This comparison illustrates why a common visual appearance does not guarantee comparable statistics. The bulk, particle, and alloy answer different questions even though they originate from the same Cu cell. Choose analysis settings according to the geometry and composition of each saved structure.

## Chapter 3

# Editing, assembly, and defects

### 3.1 Selection and edit history

Click an atom in the main scene to inspect/select it. Left-drag orbits the camera; right-drag normally pans. When **Edit > Box Select** is active, right-drag draws a selection rectangle instead. **Lasso Select** uses a right-drag path for a more irregular projected region. Selection is based on the current projection, so inspect the result from another direction before acting on a dense or overlapping set of atoms.

Open **Edit > Edit Structure** to work with the atom table. Use **Undo** and **Redo** where enabled, but retain saved checkpoints before substantial assembly or defect-generation operations. A camera reset changes only the view and does not restore a previous structure.

Ctrl+left-click adds/removes an atom from the selection. Ctrl+A selects all; Ctrl+D or Escape clears selection. Delete removes selected atoms, and right-click opens the selection context menu. Ctrl+Z undoes; Ctrl+Y or Ctrl+Shift+Z redoes. Use these shortcuts in the main workspace with no text-entry field focused.

### 3.2 Edit Structure: atoms and cell

The editor exposes cell vectors when a cell is present and an atom table with element and coordinate editing. **Position Mode** switches between **Cartesian** and **Direct**. Cartesian coordinates are in Å; direct coordinates are fractional lattice coefficients. Direct editing is disabled without a unit cell.

1. Open the source structure and save a working copy.
2. Choose **Edit > Edit Structure**. Confirm the coordinate mode before typing a position.
3. Select a row to edit the desired atom. Use the element selection and **Apply Element** controls for species changes.
4. Use **Add Atom** to create a new row, then immediately set its element and position. The initial atom is H at the origin; leaving it there may create an overlap.
5. Inspect changed positions and atom counts in the viewport and information dialog. Save the revised structure under a descriptive name.

Cell-vector edits define the simulation cell. Do not assume that typing a new box automatically performs the same operation as a homogeneous strain or a supercell construction. Check the resulting atom coordinates and boundary relationships. Use the integer transformation tool for deliberate periodic supercells.



### 3.3 Transform Structure

**Edit > Transform Structure** accepts three rows of integer coefficients. It is available for a structure with a unit cell. Begin with a diagonal transformation to establish the convention: (2, 0, 0), (0, 2, 0), (0, 0, 2) creates a  $2 \times 2 \times 2$  periodic enlargement. For conventional FCC Cu this should contain 32 atoms when the reference contains four.

Enter the matrix, choose **Apply**, and inspect the resulting lattice vectors, volume, and atom count. **Cancel** discards the pending matrix. Non-diagonal matrices define combinations of lattice directions and are useful for rotated supercells. The matrix must be nonsingular; a zero determinant cannot define a three-dimensional cell. For valid integer supercells, the determinant magnitude gives the ideal volume/atom-count multiplier before additional cuts or deduplication. Verify this count rather than assuming any arbitrary matrix represents the intended orientation.

### 3.4 Merge Structures

**Purpose.** Assemble several structures in an interactive preview. Open **Edit > Merge Structures**, load/drop the components, and select one component at a time. Its transform and gizmo controls translate or rotate that component; camera movement changes the view of the entire assembly.

1. Prepare and save each input independently with consistent length units.
2. Add the inputs to the merge dialog. Check each component's identity before moving it.
3. Select a component and use the visible gizmo handles to position/orient it. Inspect from at least two directions; two objects that appear separated in one projection can intersect in three dimensions.
4. Toggle **Show Bounding Box** to inspect the enclosure. Use **Delete Selected** to remove an unwanted component from the assembly, or **Clear All** to start the assembly again.
5. Choose **Merge Structures**, inspect the resulting scene, and save a new structure.

Merging is geometric assembly. It does not find a low-strain epitaxial match, remove all close contacts, or determine a chemically correct interface termination. Inspect separations, stoichiometry, cell conventions, and periodic contacts before using the merged structure in a simulation.

### 3.5 Cell Sculptor: supercells and cutting slabs

**Edit > Cell Sculptor** combines a periodic source, supercell repetitions, and cutting planes/slabs. Use **Use Scene** to obtain the current structure or supply a source through the dialog's load/drop workflow. The source and result previews help distinguish the uncut periodic model from the generated product.

**Procedure.** Set the supercell repetitions first. Add a cutting slab, enter nonzero Miller indices, choose periodic or distance-based limits, and inspect the plane preview. Add further slabs when the desired region needs multiple constraints. Set the outside-atom opacity to make excluded atoms less prominent, then choose **Generate**. Save the resulting atomic structure only after checking which side of each boundary was retained.

In periodic slab mode, **Start** identifies the starting plane and **N** controls the number of structural repeats/layers along the slab normal. The displayed spacing derives from the lattice. A crystallographic repeat interval can include several chemically distinct atomic layers; do not equate **N** blindly with the number of visible atomic sheets. Distance-based limits use Å and are useful for a finite cut without an integer repeat interpretation.

For a lattice with reciprocal vectors defined without  $2\pi$ ,

$$\mathbf{a}^* \cdot \mathbf{a} = 1, \quad \mathbf{a}^* \cdot \mathbf{b} = \mathbf{a}^* \cdot \mathbf{c} = 0, \quad d_{hkl} = \frac{1}{|h\mathbf{a}^* + k\mathbf{b}^* + l\mathbf{c}^*|}.$$

The normal to a plane is a reciprocal-space combination. For a skew cell it is generally not the same Cartesian direction as  $h\mathbf{a} + k\mathbf{b} + l\mathbf{c}$ . Use the preview to confirm the orientation before generating a slab. Geometric cutting can create polar surfaces or nonstoichiometric terminations; choose terminations according to the intended study.

## 3.6 Insert Dislocation

**Purpose.** Apply a continuum-style displacement construction to a structure. Open **Edit > Insert Dislocation**, use the current scene or load a reference, and use **Detect Lattice** to inspect the lattice classification before choosing directions.

**Character and region.** Choose Edge, Screw, or Mixed character. Available region shapes are Half-plane, Cylinder, Sphere, Ellipsoid, and Freeform 2D. These control the construction region. They do not by themselves certify a particular relaxed dislocation-core structure or a stable physical loop.

1. Choose a sufficiently large reference and a fixed cell geometry.
2. Enable **Auto lattice vectors** for inferred directions, or enter the slip-plane indices, Burgers direction, and line direction manually. Use crystallographically compatible, nonzero directions.
3. Locate the line using a fractional point when convenient, or the Cartesian position/offset controls. Confirm its position in the preview.
4. Set Burgers-vector scale, Poisson ratio, core radius, cutoff, and line extent. For Mixed character, set the mixed angle. Supply the selected shape's dimensions or the Freeform 2D points.
5. Choose **Generate Dislocation**. Inspect RMS and maximum displacement and the displaced structure.
6. Use **Replace Main Scene** to adopt the generated result, then save a new file.

The core radius regularises the near-core field; it is a modelling parameter, not an experimentally measured core width inferred by AtomForge. The continuum approximation is least reliable near the core. Periodic boundary conditions can require compatible dislocation arrangements; an isolated displacement construction in a periodic box may create artificial long-range strain. Relax and validate with an appropriate external model.

## 3.7 Add Interstitial Atoms and detect voids

**Purpose.** Find geometric void candidates and place chosen atoms in selected subsets. Open **Edit > Add Interstitial Atoms**, select **Use Current Scene** or provide a host, and run **Detect Voids**.

**Detection controls.** **Grid** controls the search resolution (8–30 in the current UI); **Max Voids** limits candidates. Higher resolution can find different candidates at higher cost. Use the void-type filters to include the desired coordination geometries. **Select All Types** and **Clear All Types** manage those filters. **Show Void Polyhedra in Scene** helps relate candidate centres to their coordinating atoms.

**Placement modes.**

**Random distribution** Choose a percentage or atom count and a seed. The percentage refers to eligible candidate sites, not automatically to the final atomic composition.

**Inside OBJ/STL mesh region** Load a mesh region and transform it with Translate (T), Rotate (R), and Scale (S). Inspect the mesh relative to the host; **Reset Transform** restores its transform. Choose a target percentage/count among eligible sites in that region.

**Manual (selected voids only)** Select candidate centres in the preview and check the displayed selected-centre count. The selected subset determines placement.

Use **Select Element** to choose the inserted species, then **Apply Interstitials**. Inspect the atom-count change and nearest-neighbour separations before saving. A void is a geometric candidate, not a calculated formation-energy minimum. Site preference, charge state, local relaxation, and solubility require separate physical calculations.

### 3.8 Checking an edited model

Before exporting a simulation input, check species counts, cell vectors, boundary contacts, minimum interatomic distances, and the intended periodicity. For nanoparticles and slabs, check vacuum in every relevant direction. For substitutions and defects, compare composition before and after the edit. Save both the starting model and the exact edited model used for later calculations.

## Chapter 4

# Structural analysis and presentation

### 4.1 Structure Info and Atom Info

**View > Structure Info** provides a compact check of the active structure's composition and lattice information. Use it after import, supercell construction, cutting, or merging. A disabled cell-dependent command usually means the current structure does not carry a usable cell, even when the coordinates visually resemble a crystal.

**View > Atom Info** enables atom inspection. Select the desired atom in the viewport and verify its identity and position. Displayed atom ordering follows the current structure; rebuilding or exporting can change order. Record stable identifiers in your own workflow if you need to track particular atoms across multiple files.

### 4.2 Distance and angle measurements

Use **View > Measure Distance** or the toolbar's **Distance** button, then select two atoms. For an angle, choose **Measure Angle** or **Angle**, then select three atoms in the intended order, with the middle atom as the vertex. Read the overlay and inspect the selected atoms rather than assuming the nearest projected point was selected.

Measurements are geometric. Distinguish a displayed Cartesian separation from a periodic minimum-image separation in your analysis; inspect the cell and selected image when atoms straddle a periodic boundary. For dense structures, hide unrelated species or change the camera before selection. Clear/leave the measurement mode when finished so subsequent clicks have the intended selection behaviour.

### 4.3 Radial Distribution Function

**Purpose.** Measure neighbour distributions as a function of distance. Open **Analysis > Radial Distribution Function** with a structure loaded. Select reference and target species; using different species gives a partial pair distribution. Set the distance range, bin count, and periodic-boundary option, then choose **Run RDF**.

**Controls.** Use **PBC when unit cell is available** includes periodic geometry. **Normalize to  $g(r)$**  changes from counts to the program's density-normalised distribution. **Bins** controls resolution; the current slider spans 32–2,000. **Smoothing passes** (0–8) changes the plotted/profile smoothing. **Overlay raw counts** helps distinguish sparse sampling from a smooth-looking curve. **Overlay cumulative coordination** shows the integrated neighbour count trend.

For a homogeneous isotropic system, the usual interpretation of a normalised pair distribution is

$$dN(r) \simeq 4\pi r^2 n_{\text{target}} g(r) dr.$$

Finite boxes, partial species, surfaces, and chosen normalisation affect that interpretation. A nanoparticle surrounded by vacuum does not have the same homogeneous reference density as a bulk periodic crystal. Report the species pair, bin width, cutoff, PBC choice, and smoothing when comparing curves.

**Worked check.** Open `examples/cu_host.cif`, select Cu as both species, enable PBC, and start with no smoothing. The nearest-neighbour distance of the unstrained FCC example is  $a/\sqrt{2} \approx 2.553$  Å. Check that the first peak is near that distance. This is a geometric checkpoint, not a fitted thermal distribution. A very small cell or an excessive distance range can make periodic-shell interpretation misleading.

**Distortion analysis.** Enable **Enable distortion analysis** to analyse a shell window. **Auto shell window from first-shell minima** estimates its limits; otherwise specify the visible manual bounds. Inspect the chosen window against the RDF. Overlapping shells, disorder, or low counts can make automatic minima unreliable. A broad peak can reflect multiple chemical distances, strain, temperature, or finite sampling; the plot alone does not separate those mechanisms.

## 4.4 Short-Range Order (SRO)

**Purpose.** Characterise chemical neighbourhood preferences in a multi-species structure. Open **Analysis > Short-Range Order (SRO)**, choose the number of neighbour shells and the shell-distance tolerance, and run the calculation. Inspect the species-resolved shell table.

The Warren–Cowley interpretation is

$$\alpha_{ij}^{(n)} = 1 - \frac{P(j | i, n)}{c_j},$$

where  $P(j | i, n)$  is the probability of a species- $j$  neighbour in shell  $n$  around species  $i$ , and  $c_j$  is the bulk fraction of  $j$ . Negative values indicate an excess of that neighbour relative to the reference composition; positive values indicate a deficit. Zero is the ideal random reference, subject to finite-size and composition constraints.

Use `examples/cu_ni_alloy.cif` as a starting example. The random assignment need not give exactly zero SRO in every shell. Compare several seeds or larger cells to assess sampling fluctuations. Shell tolerance groups nearby distances; too small a tolerance splits one distorted shell, while too large a tolerance merges different shells. The current shell-count control supports 1–8 shells. SRO analysis measures the existing configuration; it does not optimise an alloy toward a requested SRO value.

## 4.5 Lattice planes and Miller directions

For a periodic structure, choose **View > Lattice Planes** to configure crystallographic plane overlays, and toggle **Show Lattice Planes** for visibility. Enter the desired nonzero Miller indices and adjust the visible plane settings. Planes are guides for orientation and geometry; displaying one does not cut the structure. Use Cell Sculptor to create an actual atomic cut.

Choose **View > Miller Directions** to configure direction overlays and **Show Miller Directions** to toggle them. A direct-lattice direction  $[uvw]$  is parallel to  $u\mathbf{a} + v\mathbf{b} + w\mathbf{c}$ . A plane  $(hkl)$  is normal to  $h\mathbf{a}^* + k\mathbf{b}^* + l\mathbf{c}^*$ . In cubic cells the same integer triplet can appear parallel in both roles; this shortcut fails for a general lattice.

## 4.6 Voronoi and coordination polyhedra

**View > Show Voronoi Volume** displays a geometric nearest-site partition. Use it to inspect local space filling and site environments. It is separate from electronic Voronoi integration, which assigns scalar-grid quadrature contributions to imported sites. Neither operation is a Bader partition of electron density.

**View > Polyhedral Viewer** and the polyhedral atom representation show coordination geometry. Use **Settings > Polyhedral Settings** to configure the visible coordination construction. Check neighbour/element choices and distance criteria against known bonds in a simple reference first. A change in cutoffs can change a displayed polyhedron without any change in the underlying atoms.

**Show Dislocation Lines** controls line overlays where corresponding line information exists. An arbitrary imported XYZ file does not acquire a validated dislocation network merely by enabling the display checkbox.

## 4.7 Atomic sizes, colours, and lighting

**Settings > Atomic Sizes** lets you change display radii and restore literature defaults. Use the selected-element radius controls for a particular species or reset defaults for a consistent baseline. Display radii do not displace atom centres and should not be confused with a measured bond length.

**Settings > Display Settings** exposes lighting, material, and element-colour controls. Lighting controls change illumination; material controls change shading response. Use **Reset Lighting Defaults**, **Reset Material Defaults**, and colour reset actions to recover from an overly dark or saturated view. Avoid using visual brightness as a numerical observable.

**View > Select Theme > Light/Dark** changes the desktop theme. Electronic scalar colours and transparency are configured separately in the electronic dialog's **Appearance** and 3D controls. Use a consistent palette and numerical range when comparing several electronic fields.

## 4.8 Exporting images

Choose **File > Export Image**, select PNG, JPEG, or SVG, configure the available export settings, browse to a destination, and export. Inspect the output file at its intended presentation size. PNG is appropriate for lossless raster figures; JPEG can introduce compression artefacts around small labels; SVG is useful when a downstream tool supports the exported representation.

An image records a view, not a reproducible calculation. Accompany it with the saved structure or density, camera orientation, visible species, radii, scalar palette/range, and isovalue where relevant. For quantitative electronic profiles, export CSV and generate a plot with explicit axes and units. For the electronic dialog itself, an operating-system screenshot records both the controls and current views, as illustrated in this manual.

## 4.9 Features that are not current analysis-menu commands

Common-neighbour analysis (CNA) and angular-distribution analysis have source implementations but their menu entries are disabled in the documented baseline. They cannot be reached as ordinary Analysis commands in this executable. Their presence in source or earlier broad feature lists should not be interpreted as a tested desktop workflow. Use RDF, SRO, measurements, and the electronic tools described here for the exposed analysis features.

## Chapter 5

# Electronic files and interactive views

### 5.1 Open a scalar field

Choose **Analysis > Electronic Post-processing**. The left column contains tool selection, input files, field/operand selection, calculation parameters, export, and appearance. The right side contains the 3D and 2D views. On a smaller window, scroll the left column to reach **Calculate** and **Export**; the tool buttons remain near the top.

1. Expand **Import options** if you need to specify the physical quantity or Cube coordinate convention. Do this before loading, or use **Reload volume** after changing an import setting.
2. Click **Open volume...**. Navigate with the location shortcuts, directory list, or folder bar. Select the file and click Open, or double-click the file.
3. Check the filename, selected **Field**, dimensions, units, and displayed value range. Multi-channel files supply multiple selectable fields.
4. Choose **Charge transfer** for the eight direct charge-operation buttons or **General analysis** for the other operations.
5. Select a field and parameters, click **Calculate**, and wait for completion. Grid-producing operations append derived fields; select the intended field before the next calculation or export.

**Clear** clears the electronic session state; save useful results first. **Reload volume** rereads the source. The initial appearance estimates are based on values in the selected data, so two different files need not start with the same threshold.

### 5.2 VASP, Cube, and XSF conventions

| Input            |  | Interpretation and checks                                                                                                                                                                                      |
|------------------|--|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| VASP CHG-CAR/CHG |  | Charge-density values are normalised by the cell volume to $e/\text{\AA}^3$ . The grid is periodic without duplicate endpoint planes. Check the integrated electron count against the originating calculation. |
| VASP LOCPOT      |  | A potential field in eV. Its integral is not an electron count. Potential references and zero offsets matter when comparing calculations.                                                                      |
| VASP ELFCAR      |  | Dimensionless ELF. Choose an ELF-appropriate range; charge-only operations are not applicable.                                                                                                                 |

| Input         | Interpretation and checks                                                                                                                                                                                                                                          |
|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Gaussian Cube | Explicit coordinate convention: Bohr by default, or Angstrom. Quantity <b>Auto</b> leaves scalar values raw. Explicit density assumes atomic-unit density; explicit potential assumes Hartree and converts to eV. Coordinate and scalar-unit choices are distinct. |
| XSF datagrid  | Reads the cell/grid geometry and scalar values. Auto is conservative about scalar meaning. Specify density/potential/ELF only when the values have that meaning and the expected units. Treat finite endpoint geometry deliberately.                               |

VASP auto-detection uses conventional filenames such as CHGCAR, LOCPOT, and ELFCAR. A renamed generic file may need an explicit quantity. For Cube, negative dimension conventions vary among writers; use the explicit coordinate option rather than assuming the sign alone establishes units. Assigning a physical label to arbitrary numbers does not make them that physical quantity.

### 5.3 Units, sampling, and periodic endpoints

AtomForge’s electronic geometry uses Cartesian Å. A periodic grid with  $N_i$  samples along cell vector  $\mathbf{a}_i$  has step  $\mathbf{a}_i/N_i$  and excludes the duplicate final plane. A finite endpoint grid uses  $\mathbf{a}_i/(N_i - 1)$ . Its quadrature uses endpoint weights. These conventions affect coordinates and integration even when two files display similar shapes.

For a periodic density grid with  $N = N_a N_b N_c$  and cell volume  $V$ ,

$$Q = \int_V \rho(\mathbf{r}) dV \simeq \frac{V}{N} \sum_j \rho_j.$$

The output is an electron count when  $\rho$  is in  $e/\text{\AA}^3$ . A finite grid uses trapezoidal boundary weights. Do not sum raw values without the geometry-dependent weights.

Cube/XSF covering a complete periodic cell may include duplicate endpoint planes. **Verify periodic endpoint planes** checks opposite values and removes those duplicates when compatible. Matching endpoint values is a numerical prerequisite, not proof that the physical system is periodic. A finite molecular box with nearly zero density at its faces should not automatically be declared a periodic material.

### 5.4 What does Reference mean?

**Reference is the second input to the selected operation.** It is not a literature citation, a standard electron density supplied by AtomForge, or an automatically calculated neutral-atom density.

For subtraction, the selected **Field** is  $A$  and the reference/operand is  $B$ , giving  $A - B$ . For weighted density difference the result is  $A - wB$ . For resampling, the reference supplies the target grid geometry. For coloured isosurfaces, the reference supplies the colour values on the surface generated from  $A$ . For Boolean operations, the operand is the second binary mask.

Click **Reference...** to load another file. Then select its intended channel in the **Reference** or **Operand** selector. If no external reference is loaded, the selector can refer to a field already in the main volume. When an external reference exists, **Use loaded/result field as reference** switches the operand source back to the main loaded/derived field list. Always verify the operand after loading a new file or changing tools.



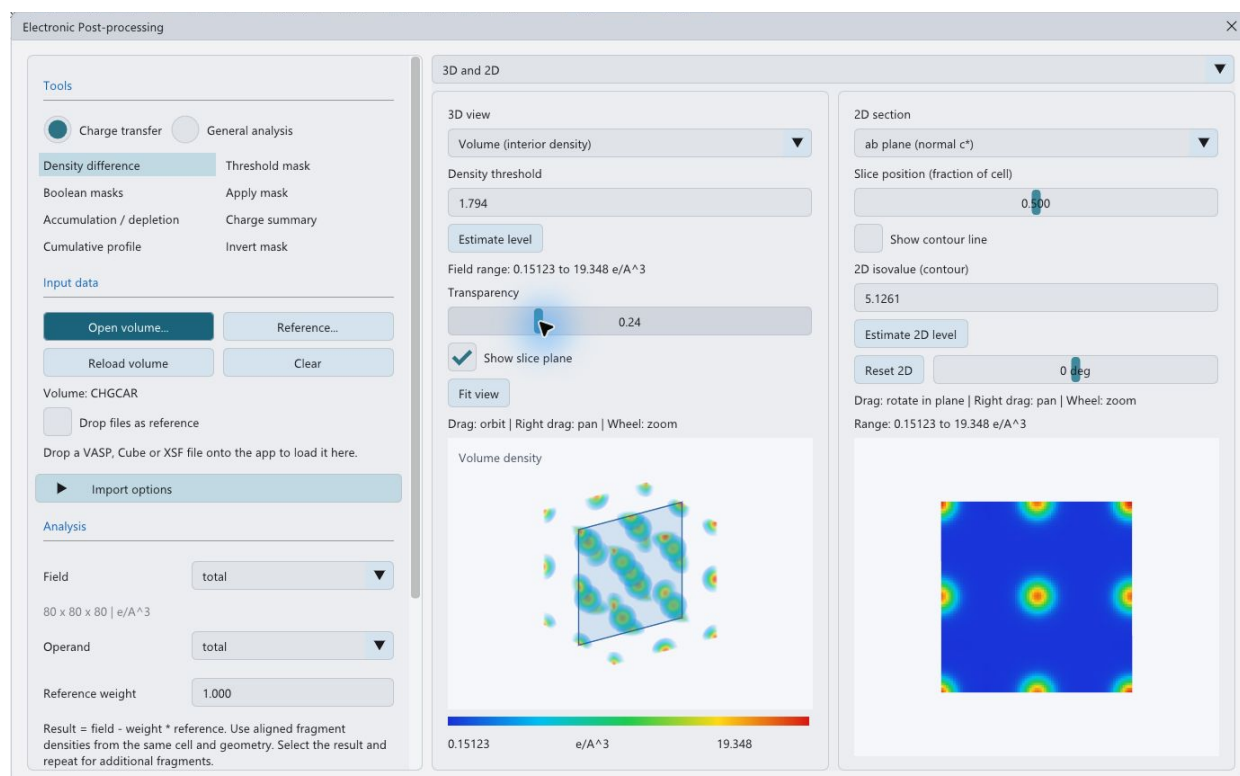


Figure 5.1: Actual AtomForge window with the supplied CHGCAR loaded. Charge-transfer tools are at the upper left. The 3D density threshold, transparency, 2D slice, and optional contour have independent controls. Transparency was reduced to approximately 0.24 for visibility in this screenshot.

### Avoid accidental self-subtraction

Immediately after loading a single-channel density, both Field and Operand can display the same field. Calculating density difference then gives zero. Load the intended independent reference, or deliberately choose another derived field, before interpreting a result as charge transfer. Self-subtraction is useful as a software sanity check only.

## 5.5 3D volume and isosurface views

Choose the layout **3D and 2D**, **Only 3D**, or **Only 2D** at the top. In the 3D panel, **Volume (interior density)** ray-integrates the sampled field above the density threshold. Lowering the threshold admits more of the field; raising it emphasises high-valued regions. This is a visual selection, not an atom/fragment partition.

**Isosurface (boundary)** extracts a surface at the chosen scalar value. After entering the level, use **Update surface**. An isosurface is a boundary of constant value; it cannot by itself show the interior as a filled volume. Choose Volume when the interior density is the intended visualisation.

**Transparency** ranges from 0 (fully opaque contributing volume/surface) to 1 (fully transparent). Even at zero transparency, regions excluded by the threshold remain empty, and a separately enabled slice guide has its own translucent appearance. Opaque does not mean that the whole cell becomes a solid cuboid.

**Camera.** Left-drag inside the 3D canvas to orbit, right-drag to pan, and use the wheel to zoom. **Fit view** or a double-click resets framing. Camera actions do not rotate the numerical grid, change its cell, or alter an exported charge integral.

**Colour and lighting.** Expand **Appearance** to choose Spectrum, Blue-white-red, or Sequential blue. Automatic range uses the data range; manual minimum/maximum supports consistent comparison. Lighting controls change shading and specular response. Colour is mapped from values, while lighting modifies their rendered appearance; use the colour scale for numerical interpretation. For signed differences, use a diverging palette with symmetric limits  $[-m, m]$  when comparing positive and negative regions.

## 5.6 Choosing an initial level and understanding warnings

The initial level is a display heuristic estimated from the file distribution: approximately the 90th percentile for a nonnegative field, zero for a signed field, with a low-range fallback for sparse fields and special handling for constant data. **Estimate level** restores the suggested 3D value. It is not an automatically determined bonding threshold or Bader dividing surface.

A 3D level outside the selected field's minimum–maximum range produces a warning and cannot generate a meaningful boundary. A 2D level is checked against the *current slice*, which can have a narrower range. A valid level in the full volume may therefore be out of range on one plane. Constant fields also cannot produce an ordinary internal contour. Enter finite numbers and check the displayed units.

Volume preview sampling is bounded at up to 96 points per axis and 256 ray steps. The interactive 2D preview uses up to 129 samples per direction. These limits keep navigation responsive; the calculation and export grids retain their own data. Use numerical sections and exported meshes/tables for analysis, and test resolution when interpreting very thin features.

## 5.7 2D lattice-plane sections

Select **bc plane (normal a\*)**, **ac plane (normal b\*)**, or **ab plane (normal c\*)**. The **Slice position** is a fraction of the corresponding cell vector. In a skew cell these normals are reciprocal directions, not necessarily Cartesian X, Y, and Z.

The filled colour map shows the sampled values. **Show contour line** is off by default; enable it to display the line at **2D isovalue (contour)**. **Estimate 2D level** chooses a suggestion from that section. Changing this contour value does not change the 3D threshold or discard the rest of the 2D colour map.

Left-drag rotates the plane's image within its own plane, right-drag pans, and the wheel zooms. The angle slider gives an explicit in-plane rotation. **Reset 2D** or a double-click resets the 2D camera. These operations leave the physical intersection plane unchanged; moving the slice position changes which data are sampled.

## 5.8 Arbitrary planes and the 3D guide

Choose **Arbitrary plane**. Enter a point as fractional  $(f_a, f_b, f_c)$  and a normal as Cartesian  $(n_x, n_y, n_z)$ . The point's Cartesian location is

$$\mathbf{p} = \mathbf{o} + f_a \mathbf{a} + f_b \mathbf{b} + f_c \mathbf{c}, \quad \mathbf{n} \cdot (\mathbf{r} - \mathbf{p}) = 0.$$

The normal must be finite and nonzero; its length does not set a thickness. For a central oblique section, use point  $(0.5, 0.5, 0.5)$  and normal  $(1, 1, 1)$ . The intersection is clipped to the actual cell. A plane that misses the cell produces a warning instead of a valid section.

Enable **Show slice plane** in the 3D controls to show the same intersection as a translucent guide. Disable it for an unobstructed density view. Rotating the 3D camera helps verify the plane

orientation; rotating the 2D image only changes its presentation. To specify a Miller-index plane in scripts, use `millers_section`, which converts the reciprocal normal explicitly.

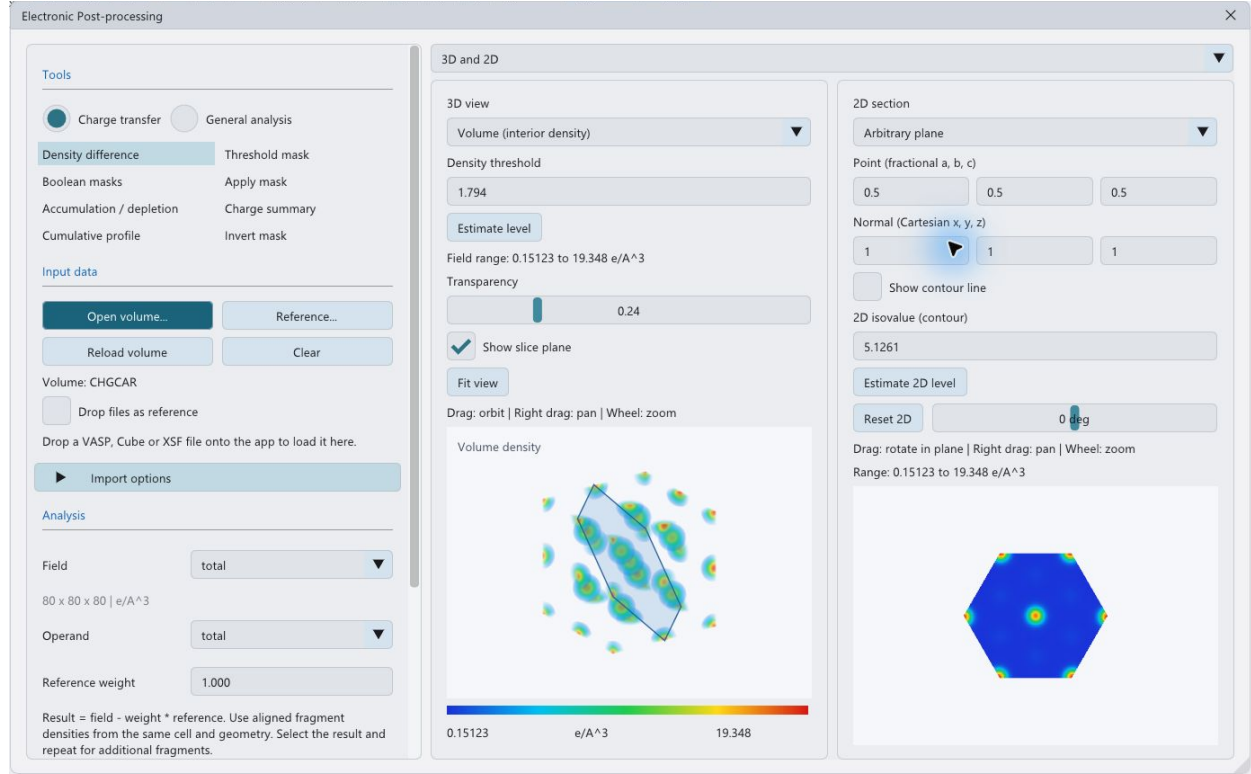


Figure 5.2: An arbitrary section of the supplied CHGCAR with a fractional point and a Cartesian normal. The translucent guide locates the section in the 3D volume.

## 5.9 Worked CHGCAR walkthrough

The documented file was supplied as `C:/Users/a.linda/Downloads/CHGCAR`. It is not copied into this repository. The first field has  $80^3 = 512,000$  samples, 16 imported sites, periodic geometry, and unit  $e/\text{\AA}^3$ . Its cell vectors in  $\text{\AA}$  are

$$\begin{pmatrix} 5.943480 & -0.002166 & -0.002166 \\ -0.002166 & 5.943480 & -0.002166 \\ -0.002166 & -0.002166 & 5.943480 \end{pmatrix}.$$

1. Open this file with Auto quantity and select the first field, **total**. Check the dimensions and unit.
2. In General analysis, calculate the total integral. Expect approximately 183.99999985 electrons for this file.
3. Inspect the ab section at position 0.5. The full field range is approximately 0.151228–19.347671  $e/\text{\AA}^3$ ; the section can contain a different subset of that range.
4. Calculate a planar average along  $c^*$  and export CSV. Then choose Charge transfer, Cumulative profile, normal  $c^*$ , Calculate, and export that table separately.
5. Check the last cumulative value against the full integral. Both use the same grid quadrature; the generated example verifies their agreement.

6. Try an arbitrary central plane, then return to the lattice plane. Save images with their thresholds and colour ranges recorded.

The approximately 184-electron total is a numerical property of the supplied first field. Without the original calculation settings and valence definitions it should not be used to infer oxidation states. The input hash and precise values are recorded in `examples/chgcar-summary.json`.

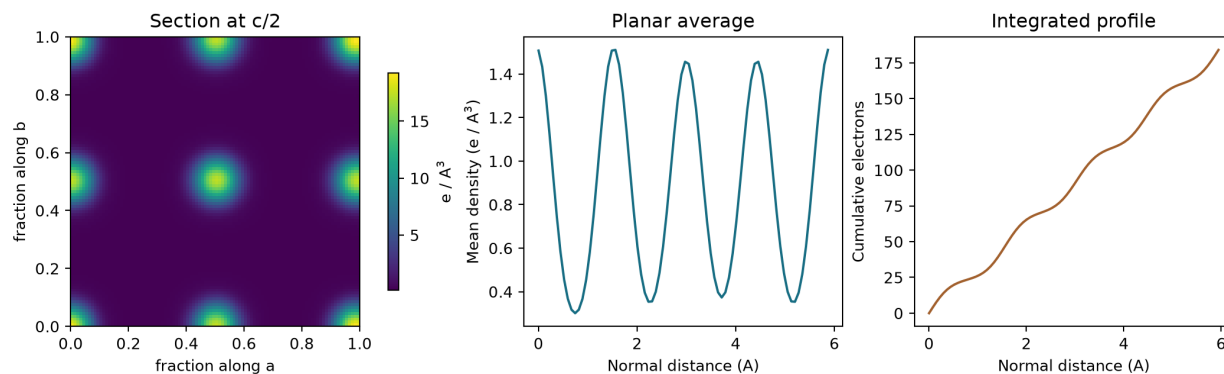


Figure 5.3: Numerical section, planar average, and cumulative integral generated through AtomForge’s Python electronic API from the supplied CHGCAR. The left panel uses fractional in-plane coordinates; the profiles use distance along the cell-face normal.

## 5.10 Exporting the right result

Expand **Export**, choose Grid: XSF/Cube/VASP, Table: CSV, or Surface: OBJ/PLY, then click **Save as...**. Browse to a directory and choose the output filename. Grid export uses the selected field; table export uses the calculated table; surface export requires a generated surface. Selecting a density field does not turn an older table into a table of that field: calculate the desired table again before exporting it.

When reopening exported grids, supply the correct quantity and endpoint convention. Generic filenames and interchange formats may not preserve enough metadata to infer physical units automatically. Cube export uses atomic-unit density/potential values; read it with the corresponding explicit quantity. Keep a record of whether duplicate periodic endpoints were written and verify round-trip geometry/integrals before combining exported data with the originals.

## Chapter 6

# Charge redistribution and Boolean masks

### 6.1 Define the study before subtracting

A common redistribution field for a combined system AB is

$$\Delta\rho(\mathbf{r}) = \rho_{AB}(\mathbf{r}) - \rho_A(\mathbf{r}) - \rho_B(\mathbf{r}).$$

Positive values mean electron accumulation relative to the chosen references; negative values mean depletion. This is an electron-density convention. The corresponding signed electric charge has the opposite sign after multiplying by the elementary charge.

References must answer the intended scientific question. For an interaction-induced difference, fragment calculations usually use positions consistent with the combined geometry, the same cell, and compatible electronic settings. Subtracting independently relaxed fragments mixes interaction and deformation effects. AtomForge does not generate fragment calculations or infer which reference is physically appropriate.

### 6.2 Grid compatibility

Before subtraction or masks, compare cell vectors, origin, dimensions, periodic/finite convention, and units. Matching atom counts or filenames is insufficient. Arithmetic requires aligned samples. A translated density with the same cell can produce a misleading difference if it is not registered consistently.

Use **Resample to reference grid** only as an explicit interpolation step when geometries are suitable. Resampling does not fix a physically inconsistent reference, and interpolation need not preserve the integral exactly. Compare integrals before and after and refine the grid when needed. Do not resample merely to silence an error without understanding what differs.

### 6.3 Weighted density difference: complete GUI recipe

1. Open CHGCAR-AB as the main volume. Select its total density field.
2. Choose **Charge transfer > Density difference**. Click **Reference...** and load CHGCAR-A.
3. Check the **Operand** channel and set **Reference weight** to 1. Click **Calculate**. The appended field represents  $\rho_{AB} - \rho_A$ .

4. Select this derived field. Load CHGCAR-B through Reference, verify that the external reference is being used, and calculate again with weight 1.
5. Select the new result  $\Delta\rho$ . Export it as a grid and use a diverging palette with an explicit symmetric colour range.
6. Calculate **Charge summary** and **Cumulative profile** from this difference field, not from the original positive density.

For more fragments, repeat the subtraction using the latest result. A weight multiplies the reference pointwise; it is useful for explicit linear combinations and must be recorded. A fractional weight does not automatically represent a physically meaningful fractional occupation calculation.

The supplied single CHGCAR cannot demonstrate a physical AB–A–B charge transfer by itself. The executable example instead checks that subtracting the file from itself produces exactly zero. Real transfer studies require the independently calculated, compatible reference densities.

## 6.4 Accumulation and depletion fields

**Accumulation / depletion** splits a density-like field into

$$\rho_+(\mathbf{r}) = \max(\Delta\rho, 0), \quad \rho_-(\mathbf{r}) = \max(-\Delta\rho, 0).$$

Both outputs are nonnegative. Depletion is stored as a positive magnitude, not as the original negative part. Select each result separately for display/export. Their difference recovers  $\Delta\rho$ ; their sum is its absolute magnitude.

A 3D positive threshold on a signed difference highlights accumulation. To see depletion with the same positive threshold, select the depletion-magnitude field. Alternatively, generate a negative isosurface on the signed difference. Record the sign, level, and colour meaning in every figure caption.

## 6.5 Charge summary

**Charge summary** reports accumulation, depletion, and net electrons:

$$Q_+ = \int \rho_+ dV, \quad Q_- = \int \rho_- dV, \quad Q_{\text{net}} = Q_+ - Q_-.$$

For references with matching total electron counts, a small net value is a useful check. A zero net value does not imply no redistribution: positive and negative regions can be substantial and cancel. Conversely, a nonzero net value can reflect electron-number choices, inconsistent normalisation, incomplete regions, or interpolation error.

These totals describe redistribution. They are not automatically the number of electrons transferred from atom A to atom B. To assign a transfer to a fragment or region, define a partition and integrate over it. Report the partition and its sensitivity alongside the result.

## 6.6 Threshold mask

**Threshold mask** creates a dimensionless binary field,

$$M(\mathbf{r}) = \begin{cases} 1, & L \leq f(\mathbf{r}) \leq U, \\ 0, & \text{otherwise.} \end{cases}$$

Bounds are inclusive, finite, and ordered. Select the source field, enter Lower bound and Upper bound in its units, and Calculate. The result has mask units and values 0 or 1. A threshold-selected region is a definition you choose, not a unique chemical boundary.

Example: a density threshold  $0.01 \leq \rho \leq \rho_{\max}$  can identify a density-supported region in a fragment field. If the lower bound lies below the entire field's range, the mask may fill the complete cell. Inspect it and its volume before interpreting a regional integral. On the supplied CHGCAR,  $0.01 \text{ e}/\text{\AA}^3$  is below the minimum and would select the whole grid with an adequate upper bound.

## 6.7 Boolean masks and inversion

Choose two aligned binary masks as Field and Operand, select **Boolean masks**, then choose an operation:

| Operation    | Retained samples                                                   |
|--------------|--------------------------------------------------------------------|
| Union        | In A or B, including their overlap.                                |
| Intersection | In both A and B.                                                   |
| Difference   | In A and not in B. Order matters.                                  |
| XOR          | In exactly one mask; excludes overlap.                             |
| Invert mask  | In the current grid domain and not in A; a single-input operation. |

To combine intermediate masks, enable **Use loaded/result field as reference** if an external reference is loaded, then select the desired derived mask as Operand. A raw density is not a binary mask and should be thresholded first. Inversion is limited to the grid domain; it does not construct the infinite space outside a molecule.

## 6.8 Apply mask and quantify a region

Select the density or difference as Field and a binary mask as Operand. Choose **Apply mask** and Calculate. Values inside the mask are retained; values outside become zero. The output retains the field's units. Select this output and calculate the total integral under General analysis to obtain the regional value.

**Example sequence.** Create fragment masks  $M_A$  and  $M_B$ , intersect them to form an overlap region, and apply this to  $\Delta\rho$ . Its integral describes net redistribution inside that particular threshold-defined overlap. Repeat with several sensible bounds to estimate partition sensitivity. The result should not be labelled a Bader charge or a unique atomic charge.

Integrating a binary mask itself gives a selected geometric volume in  $\text{\AA}^3$  under the grid quadrature. This is useful for checking whether a mask selects everything, nothing, or an unexpectedly tiny region before applying it to density.

## 6.9 Cumulative charge profile

Choose **Cumulative profile** and a plane normal  $a^*$ ,  $b^*$ , or  $c^*$ . The profile accumulates charge from the lower face of the cell to a position  $s$  along that normal:

$$Q(s) = \int_0^s \int_{\text{face}} \Delta\rho \, dA \, ds'.$$

The last row includes the entire cell and agrees with the total integral under the same quadrature. Periodic profiles include a final full-cell endpoint; finite grids use endpoint-aware integration. A cumulative electron profile is distinct from a planar-average density, whose units remain  $e/\text{\AA}^3$ .

For a slab/interface, select the actual interface normal. The location of the lower face, and therefore the cumulative curve, depends on the chosen cell origin. Moving a periodic cell boundary changes where accumulation begins. Use the same origin across comparisons and identify the interface/partition position used to quote a transfer value. A plateau can be informative, but its physical interpretation still depends on the reference and partition.

## 6.10 Recommended record for a charge study

Retain input filenames and checksums, selected channels, cell vectors/origin/grid shape, units and conversion options, all reference weights, any interpolation, and output integrals. Record mask definitions or partition geometry, profile direction and origin, 3D/2D levels, and colour scale. Verify that the integrated difference equals the corresponding weighted difference of input integrals to numerical accuracy. This check catches many bookkeeping errors before a visually convincing plot is interpreted.



## Chapter 7

# Electronic operation reference

### 7.1 How to run an operation

Load a volume, select **General analysis**, choose an operation, set the visible parameters, and Calculate. The selected Field is always the primary input. Operations producing a grid append one or more fields; operations producing a table replace the current table output. Export after verifying the selected field or table heading. The eight charge-transfer operations are detailed in Chapter 6; this chapter covers the remaining 25 operation choices.

### 7.2 Total integral

**Input:** any scalar grid. **Output:** one number, with units of field unit times  $\text{\AA}^3$ . For density this is electrons; for a mask it is selected volume; for potential it is an integrated potential, not charge. Select the field, Calculate, and export CSV if needed. Use the integral to check imports and arithmetic. The full-cell result depends on periodic/finite endpoint conventions and the determinant of the cell, not just the raw sample sum.

### 7.3 Add and subtract reference

Select A as Field and B as Reference; calculate  $A + B$  or  $A - B$ . Grid alignment and compatible units are required for meaningful addition/subtraction. These are general scalar operations. For charge studies, the dedicated weighted density-difference tool also enforces density-specific intent. Check operand order and the derived-field selection before repeating an operation.

### 7.4 Multiply and divide reference

These operations combine aligned values pointwise,  $A_j B_j$  or  $A_j / B_j$ . They are not convolution or matrix operations. Division is unsuitable where the denominator is zero or numerically close to zero; inspect the field range and error output. Dimensional meaning follows the operands, so a multiplied density does not retain the interpretation of a density integral. For region selection, use a binary mask and Apply mask, which explicitly preserves the original field unit.

## 7.5 Scale

Enter a finite **Scale factor** and Calculate. Every value is multiplied by that number. A factor of  $-1$  reverses signs; a factor of zero creates a zero field. Scaling is useful for explicitly defined linear combinations, but it does not infer the unit conversion needed by an arbitrary producer. Record any conversion factor and the original unit metadata.

## 7.6 Gaussian smoothing

Set **Sigma (A)** and **Radius (steps)**. Sigma describes the physical Gaussian width, while radius bounds the neighbourhood in grid steps. Both matter: an extremely small radius can truncate a broad Gaussian severely. The operation smooths sampled data and creates a new field; it does not increase the original physical resolution.

Start with a small kernel, inspect peaks and the integral before and after smoothing, and compare against the unsmoothed field. Finite boundaries and discrete normalisation can affect behaviour. Do not use smoothing to remove a physically meaningful oscillation without documenting it. Derivatives of a noisy field are often sensitive to smoothing and grid spacing, so report both settings.

## 7.7 Cartesian gradient

Calculate to obtain three component fields representing  $\partial f/\partial x$ ,  $\partial f/\partial y$ , and  $\partial f/\partial z$ . The derivatives account for the actual cell geometry; they are Cartesian components even when the sampling lattice is skew. Their units are field units per Å.

Select each output independently and inspect signs/ranges. A potential gradient can be related to a field only after applying the appropriate physical sign/convention. Differentiation amplifies high-frequency noise; compare resolutions and boundary behaviour. A colour map of one component does not represent the vector magnitude unless you explicitly construct that magnitude.

## 7.8 Laplacian

The Laplacian is  $\nabla^2 f$ , with units of field units per Å<sup>2</sup>. Select the field and Calculate to create a derivative grid. Positive and negative regions measure local curvature in the scalar field under the numerical discretisation.

Check a simple smooth input before interpreting fine structure. Grid resolution, smoothing, and finite boundaries can change extrema. Applying the operation to electron density does not by itself perform a full topological bonding analysis; applying it to potential does not automatically establish a charge density without the physical constants and electrostatic convention.

## 7.9 Energy-density conversion

This operation produces approximate kinetic, potential-energy, and total-energy density fields through the implemented gradient-expansion kinetic and local-virial expressions. The required input is a nonnegative electron density in  $e/\text{Å}^3$ ; outputs are in  $\text{eV}/\text{Å}^3$ . Samples at or below  $10^{-12} e/\text{Å}^3$  are masked to zero to avoid singular low-density expressions.

Select an actual density, Calculate, and inspect the three appended fields. A signed difference density is not an acceptable substitute for the required nonnegative density. The output is an approximation derived from the available scalar density. It is not the exact kinetic energy from

occupied orbitals or the complete DFT energy decomposition of the originating code. Check resolution and the effect of low-density masking before integrating or comparing these quantities.

## 7.10 Line profile

Enter **Origin (A)**, **End (A)**, and **Sample count**. The coordinates are Cartesian. Calculate to sample along the segment and obtain distance/value rows. Export CSV and plot value against distance with the selected field's units.

For a bond-axis example, obtain two atom positions and use them as endpoints, or extend slightly beyond them to inspect both sides. Finite-grid endpoints must lie within the supported domain; periodic sampling follows the periodic convention. Increasing sample count interpolates the existing grid and does not create information finer than the original calculation. Check the actual geometric length against the final profile coordinate.

## 7.11 Planar average

Choose the plane normal  $a^*$ ,  $b^*$ , or  $c^*$ . The operation averages over planes parallel to the other two cell vectors and returns normal distance and mean scalar value. A density average remains in  $e/\text{\AA}^3$ ; it is not a plane-integrated electron count.

For the  $c^*$  direction, let  $A = |\mathbf{a} \times \mathbf{b}|$  and  $L = V/A$ . Then the relationship is  $Q = A \int_0^L \bar{\rho}(s) ds$  with the corresponding numerical quadrature. This relationship is useful for interpreting the cumulative profile. In a skew cell,  $L$  is the cell height normal to the face, not necessarily  $|\mathbf{c}|$ .

## 7.12 Macroscopic average

Choose a plane normal and an odd **Window**. The operation averages the planar profile over a moving window of samples to suppress short-range oscillations. A window of one leaves the planar profile unchanged; a larger odd window spans more planes.

Relate the sample count to a physical width using the normal grid spacing. When comparing calculations with different grid sizes, the same integer window does not necessarily mean the same physical smoothing length. Inspect the unsmoothed planar average alongside the macroscopic average; broad smoothing can erase interface or surface features.

## 7.13 Numerical plane section

Enter a Cartesian origin and two non-collinear span vectors **Plane span u (A)** and **Plane span v (A)**. The sampled rectangle/parallelogram is  $\mathbf{r}(s, t) = \mathbf{o} + s\mathbf{u} + t\mathbf{v}$ , with  $s, t$  spanning the section. The GUI's sample count applies to both directions; Python accepts an explicit pair.

This is a numerical operation that creates a section field, distinct from merely changing the interactive 2D preview plane. The internal result uses two identical layers so it can be represented by the grid container. Use its section values or contours for two-dimensional analysis; integrating its artificial three-dimensional volume has no physical section-charge meaning. Finite inputs must cover the requested section, while periodic data can wrap.

## 7.14 Contour segments

Enter an isovalue and Calculate. The output is a table of Cartesian endpoints, with successive pairs defining contour segments on the source grid's first section layer. Generate/select a numerical section first when you need a particular plane. Applying contours directly to a full volume does not extract a 3D isosurface.

Choose a level within the section's range. Export CSV and connect rows in pairs rather than connecting every row as one continuous polyline. Equal-valued cells, grid resolution, and interpolation can influence topology. The interactive **Show contour line** checkbox is a display convenience; the numerical contour tool supplies coordinates for downstream work.

## 7.15 Peak search

Calculate local maxima using a 26-neighbour grid comparison. The output rows are  $x, y, z$ , value. Adjacent equal values use a deterministic tie break, so a plateau does not yield every sample as an independent peak. Python's optional limit can restrict the number returned.

Peak coordinates are based on the sampled grid; they are not sub-grid fitted critical points. Refine the grid to check location stability. Density maxima often occur near nuclei, but assigning a peak to an atom or a bond requires additional geometric and physical interpretation.

## 7.16 Voronoi integration

The GUI uses imported atomic sites and assigns sample weights to the nearest-site Voronoi regions. The result contains integral and region volume for each site in imported atom order. Python can accept explicit Cartesian partition sites. Boundary samples share weight where appropriate, and periodic geometry uses the periodic distance convention.

Check that imported sites are present and correspond to the intended structure. Compare the sum of region integrals with the full integral. The partition is geometric and sensitive to site positions. These are not zero-flux Bader basins and must not be labelled Bader charges. A region electron count also needs an explicit reference valence/nuclear convention before being converted into a net ionic charge.

## 7.17 Sphere integration

Enter a Cartesian centre in **Origin (A)** and a positive **Sphere radius (A)**. Calculate the integral over the selected spherical region according to the grid sampling and boundary convention. The radius is in Å, not fractional coordinates.

Compare several nearby radii and grid resolutions. A sphere cutting through a rapidly varying density can produce a strongly radius-dependent electron count. Check whether the sphere crosses a cell boundary and whether the finite/periodic convention matches your intended region. This tool defines a spherical partition; it does not locate an atom's natural electronic boundary.

## 7.18 Structure factors from a grid

Enter one integer `hkl` row per requested reflection. The output gives  $h, k, l, \Re F, \Im F$ . The implemented forward phase convention is positive:

$$F(\mathbf{h}) = \int_V f(\mathbf{r}) \exp(+2\pi i \mathbf{h} \cdot \mathbf{f}) dV,$$

where  $\mathbf{f}$  denotes the fractional coordinate of the Cartesian position relative to the global zero, including the grid-origin phase. Use  $F(0,0,0)$  as an integral/normalisation check. Nonzero reflections depend on origin and phase convention. Do not compare complex values from another program until those conventions agree. These Fourier tools require periodic endpoint-excluded grids and reject indices outside the unique sampled Nyquist range.

These are Fourier coefficients of the supplied field. They are not a full experimental diffraction intensity prediction including instrument response, occupancies, thermal motion, anomalous scattering, and other corrections unless those effects have been supplied by an appropriate model.

## 7.19 Fourier synthesis

Enter rows `hklrealimag`. Include conjugate pairs for a real field,  $F(-\mathbf{h}) = F(\mathbf{h})^*$ . Calculate to synthesise a field on the selected grid geometry. The inverse uses the matching phase convention and cell normalisation.

An incomplete reflection set produces a band-limited approximation and can generate termination ripples or negative values. Include the zero reflection when the mean value matters. The real-space grid must be fine enough for the highest frequencies; adding large Miller indices beyond a grid's sampling capacity does not produce a trustworthy high-resolution field.

## 7.20 Patterson map

Calculate the Patterson/autocorrelation field of a periodic input. It describes displacement-vector correlations and discards the phase information needed to uniquely recover the original field. A strong origin peak is expected for an autocorrelation and should not be mistaken for an extra atom.

Use a modest grid first and inspect runtime; reciprocal/direct transforms can be expensive. Interpret maxima as correlations of separations, not ordinary density maxima at original atom positions. Finite, incomplete, or differently normalised inputs change the meaning of the result. Preserve the original field and its units/normalisation with any exported Patterson map.

## 7.21 Ewald point-charge electrostatics

This tool requires periodic geometry, imported sites, and an explicit charge for each site in file order. Supply charges in electron-charge units. The cell must be neutral; the implementation uses a conducting boundary convention. Do not enter atomic numbers unless bare nuclear point charges are truly the intended model, and do not infer charges automatically from element names.

Set **Ewald alpha** ( $1/\text{\AA}$ ), **Real cutoff** ( $\text{\AA}$ ), and **Reciprocal cutoff** ( $1/\text{\AA}$ , includes  $2\pi$ ). Calculate. The table contains the cell energy in eV followed by the site potentials in volts. Alpha partitions the calculation between real and reciprocal space; an adequately converged result should be insensitive to reasonable alpha changes when both cutoffs are converged.

Increase both cutoffs and compare the energy and potentials before accepting a result. Ewald point-charge energy is not a self-consistent electronic total energy and does not include every energy

term of a force field or DFT calculation. Record the charge model, boundary convention, cell, alpha, and cutoffs.

## 7.22 Isosurface extraction and reference colouring

Set the 3D level and choose the Isosurface operation or **Update surface** in boundary mode. **Color from reference** samples an aligned reference field on the extracted surface. For example, a density can determine geometry while a compatible potential determines colour. Confirm both the geometric field and colour-field units.

**Append surface level** retains an existing surface and adds another level; appended surfaces must use compatible colouring units. This supports multiple boundaries but can visually obscure one another. Set transparency deliberately and export the mesh when required. OBJ stores scalar comments and PLY stores a scalar vertex property; downstream tools may need an explicit colour mapping and may not automatically reproduce AtomForge's palette. Surface vertices are emitted as triangles without welding.

## 7.23 Resample to reference grid

Select the source as Field and the desired target geometry as Reference. Calculate an interpolated source field sampled on that reference geometry. The reference's scalar values are not subtracted or blended by this operation. Check that the target points lie in the valid source domain for a finite source.

Compare output shape, cell, origin, and periodicity with the target, and compare integrals and extrema with the source. This is interpolation, not a conservative charge-remapping guarantee. Resampling is useful for explicit alignment of compatible grids but cannot repair inconsistent physical calculations.

## 7.24 Verify periodic endpoint planes

Use this only for finite Cube/XSF-style grids that span a full periodic cell with duplicated endpoints. The operation checks opposite planes and removes the duplicates, giving periodic sampling. A mismatch is a reason to inspect the input rather than force conversion. Python exposes the tolerance, default  $10^{-8}$ .

After conversion, check the shape (one fewer sample along each duplicated direction), cell, periodic flag, and integral. Re-exported periodic data can again acquire explicit endpoint planes in a finite-grid format. Record the convention at every conversion step to avoid an extra plane being counted as independent data.

## Chapter 8

# Python and command-line integration

### 8.1 Install the Python package

From the repository root, install the local package into the Python environment you intend to use:

```
python -m pip install ./python
```

The structure API uses Python implementations of its supported file formats. Electronic operations additionally require the compiled native electronic library. Set `ATOMFORGE_PATH` to the viewer executable when it is not discoverable. If necessary, set `ATOMFORGE_ELECTRONIC_LIBRARY` to the absolute native library path before importing/using electronic operations. Keep required runtime DLLs beside the Windows library/executable or in the appropriate loader search path.

```
import os
os.environ["ATOMFORGE_PATH"] = r"C:\path\to\AtomForge.exe"
# Only needed when automatic native-library discovery is insufficient:
# os.environ["ATOMFORGE_ELECTRONIC_LIBRARY"] = r"C:\path\to\atomforge_electronic.dll"
import atomforge as af
```

Inspect the actual library filename produced by your platform/build instead of assuming every platform uses the Windows prefix and suffix. A Python package installation alone does not compile or bundle every native dependency.

### 8.2 Structure objects and file I/O

`af.load(path)` returns a `Structure`. `structure.atoms` contains `Atom` objects with element symbols and Cartesian coordinates. `structure.cell` contains the three lattice vectors when available. `len(structure)` returns the atom count. Save with `structure.save(path)` or the package-level save function.

```
from pathlib import Path
import atomforge as af

examples = Path("docs/manual/examples") # run from repository root
cu = af.load(str(examples / "cu_fcc.cif"))
assert len(cu) == 4
host = cu.repeat(5, 5, 5)
assert len(host) == 500
host.save("cu_host.cif")
```

```
edited = host.copy()
edited.translate(0.1, 0.0, 0.0) # in place; cell is unchanged
edited.scale(1.01) # in place; atoms and cell scale
edited.save("cu_shifted_scaled.cif")
```

`repeat(nx,ny,nz)` returns a new structure and requires positive integers and a cell. `copy()` makes a separate structure. `filter_species("Cu","Ni")` returns a filtered copy. `add_atom(symbol,x,y,z)` appends a Cartesian site; `remove_atom(index)` removes one by index. `set_cell(a,b,c,alpha,beta,gamma)` defines a lattice from lengths and angles. Check indexing after removals because later indices shift.

```
s = af.Structure()
s.set_cell(8.0, 8.0, 8.0, 90.0, 90.0, 90.0)
s.add_atom("Fe", 2.0, 4.0, 4.0)
s.add_atom("C", 3.8, 4.0, 4.0)
s.save("illustrative_pair.xyz")
# This is an illustrative geometry, not a relaxed Fe-C molecule.
```

The Python structure codecs cover XYZ and extended XYZ, VASP files, PDB, CIF with explicit P1 sites, and LAMMPS data. This is not identical to the desktop's Open Babel coverage. The Python CIF path does not replace the bulk builder's space-group expansion: use already expanded sites when importing a symmetry-rich CIF through this API. LAMMPS restricted-triclinic conversion can rotate geometry and shift origins; verify the resulting cell in the consuming simulator.

### 8.3 Desktop and notebook viewing

`structure.view()` opens the structure in AtomForge without blocking the Python workflow. `af.view(structure)` provides the package-level viewer. `structure.view_notebook(width=620,height=460)` requests an inline notebook view; if IPython support is absent it falls back to the desktop viewer. The notebook representation is a visual aid and does not run an electronic calculation.

```
host.view()
# In a Jupyter notebook:
host.view_notebook(width=800, height=600)
```

### 8.4 Loading electronic volumes

```
from atomforge.electronic import load_volume

volume = load_volume("CHGCAR")
rho = volume.fields[0]
print(rho.shape, rho.cell, rho.origin, rho.periodic, rho.unit)
print(len(rho.sites), rho.integrate())

# Cube coordinates and scalar quantity are separate choices:
cube = load_volume("density.cube", quantity="density",
                  cube_coordinates="bohr").fields[0]
xsf = load_volume("density.xsf", quantity="density").fields[0]
```

`Volume.fields` is a collection of scalar channels. `Grid.values` is a flat sequence with x varying fastest, then y, then z: index  $x + N_x(y + N_y z)$ . If using NumPy, reshape to  $(nz, ny, nx)$  for



conventional  $z/y/x$  array indexing. `Grid.sites` contains rows of atomic number and Cartesian  $x, y, z$ . Derived grids generally return new objects; preserve names in your script so that the intended intermediate is explicit.

## 8.5 A complete scripted charge workflow

The following recipe requires three independently calculated, aligned density files. It is not executed using fabricated fragment densities in this manual.

```
import csv
from atomforge.electronic import load_volume

total = load_volume("CHGCAR-AB").fields[0]
a = load_volume("CHGCAR-A").fields[0]
b = load_volume("CHGCAR-B").fields[0]
delta = total.density_difference(a, b, weights=[1.0, 1.0])
delta.save("AB_minus_A_minus_B.xsf")

summary = delta.charge_summary()
print(summary["accumulation"], summary["depletion"], summary["net"])
accumulation, depletion = delta.split_density()
accumulation.save("accumulation.xsf")
depletion.save("depletion_magnitude.xsf")

mask_a = a.threshold_mask(0.01, max(a.values))
mask_b = b.threshold_mask(0.01, max(b.values))
overlap = mask_a.boolean(mask_b, "intersection")
regional = delta.apply_mask(overlap)
print("Overlap redistribution, electrons:", regional.integrate())

with open("cumulative.csv", "w", newline="") as stream:
    writer = csv.writer(stream)
    writer.writerow(["distance_A", "cumulative_e"])
    writer.writerows(delta.cumulative_charge(axis=2))
```

Use `boolean(..., "union")`, `"difference"`, or `"xor"` for the other mask combinations. `invert_mask()` complements a binary mask. `density_difference` accepts multiple references and one optional weight per reference. Array operators support aligned arithmetic; scalar multiplication/division support explicit scaling. Check units and division denominators.

## 8.6 Profiles, sections, derivatives, and integration

```
gx, gy, gz = rho.gradient()
lap = rho.laplacian()
smoothed = rho.smooth(sigma=0.2, radius=2)
kinetic, potential, total_energy = rho.energy_density()

profile = rho.line_profile((0, 0, 0), (1, 1, 1), count=100)
planar = rho.planar_average(axis=2)
macro = rho.macroscopic_average(axis=2, window=5)
q_sphere = rho.integrate_sphere(center=(1, 1, 1), radius=0.5)
site_rows = rho.voronoi_integrate() # imported sites, file order
peaks = rho.peaks(limit=10)
```

```
origin = tuple(rho.origin[i] + 0.5*rho.cell[2][i] for i in range(3))
section = rho.section(origin, rho.cell[0], rho.cell[1], (100, 100))
values_2d = section.section_values
segments = section.contours(level=1.0)
```

Coordinate examples above assume those locations lie in the input domain. The returned section is a two-layer container; use `section_values` for 2D data. Returned profile and integration tables are Python row sequences suitable for CSV or plotting. Choose a contour level inside the section range rather than blindly reusing 1.0 for every file.

`miller_section(grid, hkl, offset, width, height, shape)` constructs an orthonormal section with a reciprocal-space normal. Offset defines the fractional equation  $hf_a + kf_b + lf_c = \text{offset}$ ; width and height are in Å. The in-plane orientation is selected deterministically. For finite data the entire requested rectangular section must lie inside the supported field.

## 8.7 Fourier and scattering models

`structure_factors(indices)` returns reflection rows. `fourier_synthesis(reflections)` creates a field from explicit complex reflection rows. Both require periodic endpoint-excluded geometry, unique sampled Miller indices, and consistent phase conventions. `patterson()` returns the periodic autocorrelation field.

```
reflections = rho.structure_factors([(0, 0, 0), (1, 0, 0), (-1, 0, 0)])
low_frequency = rho.fourier_synthesis(reflections)
low_frequency.save("three_reflections.xsf")
```

This deliberately incomplete synthesis retains only three Fourier terms and is not a reconstruction of the full original field. The zero reflection should equal the integral. Include conjugate pairs explicitly; invalid duplicates, out-of-range reflections, and a complex-valued result are rejected.

For atom-based scattering calculations, `atomic_structure_factors` accepts an explicit complex scattering value for every reflection/site, plus optional occupancies and isotropic B factors in Å<sup>2</sup>. Occupancies are 0–1 and B factors nonnegative. No oxidation states or element coefficient tables are inferred.

`scattering_factor(s, a, b, c)` evaluates  $\sum_i a_i \exp(-b_i s^2) + c$ , with  $s = \sin \theta / \lambda$ . Supply a valid coefficient source and respect its applicable range. `model_density` requires exactly one mapping keyed by atomic number: X-ray coefficient tuples or neutron scattering lengths. It constructs a finite Fourier model on periodic geometry. Even-grid Nyquist planes are omitted to retain a real conjugate-symmetric expansion. These free-atom/scattering models are not self-consistent DFT charge densities and should not be substituted silently for fragment reference calculations.

## 8.8 Point charges, surfaces, and explicit geometry

`rho.ewald(charges, alpha, real_cutoff, reciprocal_cutoff)` returns a dictionary with energy in eV and potentials in volts. Charges must match the imported sites and sum to a neutral cell. Verify convergence using both cutoffs.

`rho.isosurface(level, color=other_grid)` returns a Surface. Its `vertices` property provides triangle/vertex data; `save("surface.obj")` or `save("surface.ply")` writes mesh output. Reference colouring requires compatible geometry and meaningful units. A scalar property in an exported mesh may require colour-map configuration in the next application.

```
from atomforge.electronic import Grid
```

```

geometry = Grid((8, 8, 8), ((4, 0, 0), (0, 4, 0), (0, 0, 4)),
               [0.0] * (8*8*8), periodic=True, unit="e/A^3")
# Attach atomic-number / Cartesian-coordinate sites explicitly:
geometry = geometry.with_sites([(1, 0, 0, 0), (1, 2, 2, 2)])
result = geometry.ewald([1.0, -1.0], alpha=0.8,
                       real_cutoff=12.0, reciprocal_cutoff=8.0)
print(result)

```

This is an illustrative neutral point-charge model. The two H labels supply site metadata; the explicitly supplied  $+1, -1$  charges define the electrostatic model. It is not a prediction for a real hydrogen compound.

## 8.9 Electronic CLI

The Python module offers conversion and selected numerical operations without opening the GUI:

```

python -m atomforge.electronic CHGCAR --operation info
python -m atomforge.electronic CHGCAR --operation integrate
python -m atomforge.electronic CHGCAR --operation planar --axis 2 --output planar.csv
python -m atomforge.electronic CHGCAR --operation isosurface --level 1.8 --output
    density.ply
python -m atomforge.electronic density.cube --quantity density --operation convert --
    format xsf --output density.xsf

```

Choose channels with `--channel` (zero-based). Use `--quantityauto|raw|density|potential|elf` and `--cube-coordinatesbohr|angstrom`. `--periodic` verifies and removes duplicate endpoints; it is not a generic instruction to make arbitrary data periodic. Operations other than `info/integrate` require an output. Additional choices include `laplacian`, `patterson`, `kinetic`, `potential-energy`, `total-energy`, and `peaks`. Charge-mask combinations are available through Python scripts and the GUI, not every one as a dedicated CLI choice.

## 8.10 Native builder CLI

`AtomForge--buildMODE--help` prints the mode's accepted options. Available documented modes are `bulk`, `sss`, `gb`, `nano`, `poly`, `amorphous`, and `custom`. Each returns an output structure without requiring a GUI interaction. Use a subprocess argument list from Python to preserve spaces and avoid shell quoting problems:

```

import os
import subprocess
subprocess.run([
    os.environ.get("ATOMFORGE_PATH", "AtomForge"), "--build", "bulk",
    "--system", "cubic", "--spacegroup", "225", "--a", "3.61",
    "--atom", "Cu 0 0 0", "--output", "cu_fcc.cif"
], check=True)

```

Check the return code and read error output on failure. Keep the exact input arguments and seed with generated files. The manual's example generator uses this pattern and verifies every produced structure can be loaded with a nonzero atom count.

## Appendix A

# Installation, builds, and troubleshooting

### A.1 Running a packaged desktop build

Extract a complete package into a writable folder and run the executable. Keep the bundled runtime libraries and Open Babel plugins in their supplied locations. Copying only `AtomForge.exe` to another machine can leave required DLLs or format plugins behind. To open a structure at launch, pass its path as the executable's argument.

The desktop requires a working OpenGL environment. If the window fails to render, check the graphics driver and whether a remote/virtual session supplies the needed OpenGL support. A successful compiler run and a working graphics runtime are separate checks.

### A.2 Build requirements

The source uses C++17 and CMake 3.16 or newer. Desktop dependencies include GLFW, GLEW, GLM, Open Babel 3, and an OpenGL implementation. spglib enables symmetry-related capabilities. Use a consistent compiler/runtime family for all linked libraries. See the repository's `INSTALL.md` and actual CMake configuration for platform-specific dependency discovery.

| CMake setting                                                                 | Purpose                                                                                               |
|-------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| <code>CMAKE_BUILD_TYPE=Release</code><br><code>ATOMFORGE_BUILD_APP=OFF</code> | Optimised application/library build.<br>Build core/library functionality without the desktop app.     |
| <code>BUILD_TESTING=ON</code><br><code>ATOMFORGE_ENABLE_SSS_BUILDER=ON</code> | Configure available automated tests.<br>Enable the substitutional-alloy desktop builder (default on). |
| <code>ATOMFORGE_ENABLE_SFE_BUILDER=ON</code>                                  | Enable the optional stacking-fault desktop builder (default off).                                     |
| <code>BUILD_PORTABLE=ON</code><br><code>Python3_EXECUTABLE</code>             | Enable runtime packaging support.<br>Select the Python interpreter used by configured tests.          |

## A.3 Linux build

Install a C++ toolchain, CMake, pkg-config, and development packages for GLFW, GLEW, GLM, Open Babel, and optionally spglib using your distribution's package manager. Configure a fresh build directory:

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release -DBUILD_TESTING=ON
cmake --build build -j
ctest --test-dir build --output-on-failure
./build/AtomForge
```

Run from a graphics-capable session for desktop use. For a headless server, use the native builder CLI or electronic library/API instead. A core-only build does not create a desktop executable.

## A.4 Windows build with MSYS2 UCRT64

Use the MSYS2 UCRT64 toolchain and its matching dependency packages. Typical packages include the UCRT64 toolchain, CMake, GLFW, GLEW, GLM, Open Babel, pkgconf, spglib, and Python when running the Python tests. Avoid mixing MINGW64, UCRT64, and unrelated compiler libraries in one build directory.

```
cmake -S . -B build -G "MinGW Makefiles" -DCMAKE_BUILD_TYPE=Release -DBUILD_TESTING=ON
-DPython3_EXECUTABLE=C:/msys64/ucrt64/bin/python.exe
cmake --build build -j
ctest --test-dir build --output-on-failure
```

These commands assume the compiler and matching libraries are discoverable, normally from the UCRT64 shell. If using PowerShell, supply the correct toolchain paths and runtime DLLs. Select a Python interpreter compatible with the native library's runtime requirements for tests; discovering an unrelated system Python can make a correct library build appear to fail during loading.

Use a new build folder when changing generators or compiler families. Close a running AtomForge instance before replacing its executable, because Windows may lock the file. After rebuilding, launch the executable from the expected folder and check that the interface matches the new version; an older shortcut can launch a stale copy.

## A.5 Portable packaging and CI

Configure with `BUILD_PORTABLE=ON`, build, then use CPack with that build directory's `CPackConfig.cmake`. Validate the extracted package in a clean runtime environment and test an actual structure import, not only process startup. Open Babel plugins are needed for format support.

When a GitHub compilation job fails, inspect the first failing step and its log: configure/dependency discovery, compilation, linking, tests, or packaging. The last generic failure line is usually less useful than the original diagnostic. Record the commit and platform. Documentation changes do not resolve an unrelated toolchain problem, and a locally successful Windows run does not alone prove every CI platform passes.

## A.6 Reporting a reproducible problem

Record the application commit, operating system, build options, input/import settings, selected fields, exact steps, and error text. Supply a small reproducer with expected and observed behaviour.

For numerical issues include geometry, units, and integrals; for visual issues capture the entire dialog.

## A.7 Troubleshooting by symptom

| Symptom                                 | Checks and recovery                                                                                                                                                                   |
|-----------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Charge tools not visible                | Open Electronic Post-processing and select Charge transfer. The eight tool buttons should appear before loading a file. Check that the expected rebuilt executable is running.        |
| Calculate/Export not visible            | Scroll the left control pane; long parameter sets extend below the initial view. Resize the dialog if needed.                                                                         |
| Reference gives zero difference         | Field and Operand may be the same channel. Verify the external reference file and the local-reference checkbox.                                                                       |
| Grid alignment error                    | Compare shape, cell, origin, units, and periodicity. Resample explicitly only if scientifically justified.                                                                            |
| Charge operation rejects input          | Inspect units. Cube/XSF Auto is raw; choose the correct quantity and reload. A potential/ELF/mask is not electron density.                                                            |
| Wrong integral after conversion         | Check scalar-unit conversion, volume normalisation, coordinate units, duplicate endpoints, and selected channel.                                                                      |
| 3D looks empty                          | Check threshold/range, transparency, selected field, render mode, and Fit view. For a signed field, select depletion magnitude to display negative changes with a positive threshold. |
| Opaque slider still shows empty regions | Threshold-excluded space remains empty. Hide the separately translucent slice plane if it obscures interpretation.                                                                    |
| Unexpected circle/line in 2D            | Check Show contour line. It is optional and off by default. It does not control the filled colour map.                                                                                |
| 2D out-of-range warning                 | Compare the level with the current slice's range; Estimate 2D level or move the plane.                                                                                                |
| Arbitrary section absent                | Use a finite nonzero normal and a point that defines a plane intersecting the cell.                                                                                                   |
| Fourier operation rejected              | Use periodic endpoint-excluded geometry, unique indices within the sampled Nyquist range, and conjugate pairs for real synthesis.                                                     |
| Fourier workload too large              | The implementation bounds work. Consider an explicitly resampled power-of-two grid and quantify the interpolation effect before continuing.                                           |
| Ewald rejected or unstable              | Check neutrality, site/charge count and order, periodicity, positive parameters, and convergence of both cutoffs.                                                                     |
| Python native DLL not found             | Set the native-library path and make its dependent runtime DLLs available. Confirm process/library architecture compatibility.                                                        |
| Builder menu disabled                   | Load a suitable source with a cell; inspect compile-time options and symmetry support.                                                                                                |
| Mesh fill missing atoms                 | Check closed geometry, units/scale, replication coverage, and feature thickness relative to the lattice.                                                                              |
| Amorphous placement fails               | Density, pair cutoffs, and counts may be incompatible. Test a looser geometry and inspect diagnostics.                                                                                |
| Saved structure loses cell/metadata     | Choose a format supporting the needed fields and verify the re-opened result. Python and desktop format coverage differ.                                                              |

## Appendix B

# Examples, validation, and maintaining this manual

### B.1 Included example inventory

The following structures were generated through AtomForge’s actual builder CLI, with the larger Cu host produced through the Python repetition API. They are reusable inputs for the walkthroughs.

| File               | Atoms | Construction                                                                       |
|--------------------|-------|------------------------------------------------------------------------------------|
| cu_fcc.cif         | 4     | Cubic 225, Cu at the origin, $a = 3.61 \text{ \AA}$ .                              |
| fe_bcc.cif         | 2     | Cubic 229, Fe at the origin, $a = 2.87 \text{ \AA}$ .                              |
| cu_host.cif        | 500   | $5 \times 5 \times 5$ repetition of the Cu conventional cell.                      |
| cu_ni_alloy.cif    | 500   | Cu 0.7 / Ni 0.3, seed 42.                                                          |
| cu_sphere.xyz      | 336   | Sphere radius $10 \text{ \AA}$ , vacuum $5 \text{ \AA}$ .                          |
| cu_sigma5.cif      | 60    | Axis [001], $\Sigma 5$ , candidate 0, repetitions 3/3, overlap $1.5 \text{ \AA}$ . |
| cu_polycrystal.cif | 1,329 | $25^3 \text{ \AA}^3$ , four grains, seed 7.                                        |
| sio2_pack.xyz      | 120   | Si 40 / O 80, density $1.5 \text{ g cm}^{-3}$ , seed 7.                            |
| cu_mesh.xyz        | 224   | Unit octahedron OBJ at scale $12 \text{ \AA}/\text{model unit}$ .                  |

octahedron.obj is an original procedural mesh included with the examples. The builder gallery plots actual output coordinates with Matplotlib. No external 3D model or Blender-generated asset is needed to reproduce these examples.

### B.2 CHGCAR provenance

The original file remains at the user-supplied location and is not redistributed. `chgcar-summary.json` records its SHA-256 hash, dimensions, cell vectors, sites, units, range, integral, and periodicity. The manual includes derived `chgcar-planar.csv` and `chgcar-cumulative.csv` plus figures. A different CHGCAR will generally produce different numbers and images; the filename alone is not a unique identifier.

The example generation verifies that self-subtraction is exactly zero and that the final cumulative integral agrees with the full integral. These are numerical consistency checks, not independent validation of the originating electronic-structure calculation or a physical charge-transfer study.

Run `pythondocs/manual/scripts/check_examples.py` to verify the published structure counts and executable Python recipes. It uses an analytic periodic field to check charge conservation under

complementary masks, Fourier reconstruction, derivative outputs, integration, neutral point-charge calls, and Cube/XSF round trips. This synthetic fixture is explicitly a software check and is separate from the supplied CHGCAR walkthrough.

## B.3 Regenerate examples and figures

Build AtomForge and install the local Python package/native library first. The plotting script additionally needs NumPy and Matplotlib. Run from the repository root:

```
python -m pip install numpy matplotlib
python docs/manual/scripts/generate_examples.py --chgcars "C:/path/to/CHGCAR"
```

Omit `--chgcars` to regenerate only structure examples and the builder gallery. Set `ATOMFORGE_PATH` if the executable is outside the normal build folder. The optional `MANUAL_PYTHON_DEPS` variable can point to a separate plotting-package directory when using an isolated embedded Python runtime.

The script overwrites the generated examples and scientific figures in this manual folder. Review new counts and the provenance JSON after changing software versions. It does not automate GUI screenshots: recapture those from the actual application when controls change. Keep scientific plots and interface captures labelled distinctly.

## B.4 Compile the LaTeX manual

Install a LaTeX distribution providing pdfLaTeX and the packages used by `manual.tex`: Latin Modern, microtype, geometry, graphicx, xcolor, booktabs, longtable, tabularx, amsmath, enumitem, listings, fancyhdr, tcolorbox, and hyperref. Then run:

```
python docs/manual/scripts/build_manual.py
```

The script compiles three passes for contents, page references, and PDF bookmarks. It places intermediates in `docs/manual/build/` and the distributable PDF at `docs/manual/AtomForge-manual.pdf`. The `build/` folder is ignored by Git. No raw CHGCAR is required to compile the checked-in manual because its published figures are included.

After editing, inspect the compiler log for overfull boxes, missing references, and missing characters. Render the PDF to page images and inspect figures, captions, tables, code wrapping, and page breaks. A successful LaTeX exit code alone does not establish a readable document. Keep screenshots at sufficient resolution for labels to remain legible at the chosen page width.

## B.5 Documentation coverage and limitations

The manual covers the exposed File/Edit/Build/View/Analysis/Settings workflows, all 33 electronic operation choices, Python structure and electronic APIs, and CLI entry points. Optional stacking-fault controls are identified separately; unavailable interface/CNA/ADF menus are explicitly distinguished from exposed features. Screenshots cover representative builder and electronic workflows rather than every possible parameter combination.

The examples test executable generation and numerical consistency. They do not constitute convergence studies for every builder, material, or physical model. In particular, the single supplied CHGCAR does not establish a fragment-transfer reference set; random structures are not relaxed glasses; and model scattering densities are not self-consistent electronic densities.



## B.6 Source map for maintainers

| Source area                                      | Documentation to review when it changes                                        |
|--------------------------------------------------|--------------------------------------------------------------------------------|
| <code>src/ui/FileBrowser.cpp</code>              | Menu availability, file workflows, shortcuts, help and settings placement.     |
| <code>src/ui/*BuilderDialog.cpp</code>           | Builder controls, defaults, validation, preview/commit steps.                  |
| <code>src/algorithms/</code>                     | Builder geometry and structural-analysis assumptions.                          |
| <code>src/ui/ElectronicPostProcessing.cpp</code> | Tool labels, operand routing, imports/exports, calculation parameters.         |
| <code>src/ui/ElectronicSliceViewport.cpp</code>  | 2D controls, arbitrary-plane behaviour, preview limits.                        |
| <code>src/electronic/</code>                     | Units, endpoint conventions, numerical operations, Fourier and electrostatics. |
| <code>src/graphics/ElectronicViewport*</code>    | Rendering, opacity, lighting, volume preview behaviour.                        |
| <code>python/atomforge/</code>                   | Python signatures, supported formats, library discovery, CLI choices.          |
| <code>CMakeLists.txt</code> , CI workflows       | Feature flags, dependencies, tests, packaging and interpreter selection.       |

Update the source baseline on the title page when revising the manual. Re-run examples after algorithm or format changes, compare the generated metadata, and recapture affected UI figures. Avoid describing a feature as available solely because a corresponding source file exists.

## B.7 Glossary

**Asymmetric unit** Independent crystallographic sites expanded by space-group operations.

**Cell / origin** Lattice vectors and starting Cartesian position defining the grid or structure domain.

**Direct coordinates** Dimensionless coefficients of the direct lattice vectors.

**Field / channel** One scalar quantity sampled on a grid, such as total density or a derived difference.

**Reference / operand** The explicitly selected second input to an operation.

**Isosurface** A three-dimensional boundary at a constant scalar value.

**Contour** A constant-value line on a two-dimensional section.

**Volume rendering** Accumulation of contributions along viewing rays through a scalar field.

**Mask** Binary field defining a region within a grid domain.

**Reciprocal normal** A normal constructed from reciprocal lattice vectors; distinct from a direct-lattice direction in a general cell.

**Periodic endpoints** Repeated boundary samples that must be distinguished from unique periodic samples.

**Charge redistribution** Density change relative to specified references; not automatically a unique atom-assigned transfer.

**Voronoi partition** Geometric nearest-site regions, distinct from density-gradient basins.